

Introduction à la conception optimale pour des systèmes régis par EDP

Régis Duvigneau

Septembre 2020

1 Définition du problème

On supposera dans l'ensemble de ce cours qu'on cherche à résoudre un problème de la forme suivante :

$$\begin{aligned} &\text{Minimiser } j(x, w(x)) \\ &\text{Soumis à } \psi(x, w(x)) = 0 \end{aligned} \tag{1}$$

où j représente une fonctionnelle coût, x le vecteur des variables de contrôle de dimension n et w le vecteur des variables d'état du système. Celui-ci est déterminé par les équations d'état ψ , considérées comme contraintes pour le problème.

Dans un context d'ingénierie, ce problème peut designer, par exemple, un problème d'optimisation de forme ou de contrôle optimal. Les équations d'état ψ représentent alors le système d'équations aux dérivées partielles, par exemple les équations de Navier-Stokes (mécanique des fluides) ou d'élasticité (mécanique des structures) et w représente les variables d'état, comme le champ de vitesse, pression, déplacement, etc. Pour donner quelques exemples concrets, un problème typique en optimisation de forme aérodynamique est la réduction de traînée d'un profil d'aile. La fonction de coût prend alors la forme suivante :

$$j(x, w(x)) = \int_{\Gamma(x)} \overline{\overline{T}}(w(x)) \cdot \vec{n}(x) d\Gamma \cdot \vec{X} \tag{2}$$

où Γ représente la forme du profil paramétrisée par les variables de forme x , $\overline{\overline{T}}$ le tenseur des contraintes dépendant des variables d'écoulement w , $\vec{n}$ la normale au profil et $\vec{X}$ la direction de l'écoulement amont. Dans le contexte du contrôle optimal des écoulements, la fonction de coût prend typiquement la forme suivante :

$$j(x, w(x)) = \frac{1}{2} \int_{T_i}^{T_f} \|w(x) - w^*\|^2 dt \tag{3}$$

où T_i et T_f sont les bornes d'un intervalle de temps, pendant lequel on souhaite contrôler les variables d'écoulement w pour qu'elles atteignent l'écoulement cible caractérisé par w^* . La formulation du problème (1) est donc suffisamment générale pour les applications qui nous intéressent.

2 Classification des approches possibles

Les méthodes d'optimisation (les plus courantes) en ingénierie peuvent être classées en plusieurs catégories :

- **Les approches déterministes :**

- Les méthodes de descente

Elles consistent à définir une suite de vecteurs des variables de contrôle minimisant progressivement la valeur de la fonction coût, en se déplaçant à chaque itération selon une direction de recherche définie à partir des dérivées de la fonction de coût. Il s'agit principalement des méthodes de plus grande descente, de Newton et quasi-Newton[6].

- Les méthodes de type simplexe

Elles consistent à évaluer la fonction de coût en un ensemble de $n + 1$ vecteurs des variables de contrôle (formant un simplexe dans $\mathbb{R}^n$), puis à déplacer cet ensemble dans l'espace des variables de contrôle selon les résultats obtenus. Ces approches ont été proposées par Spendley, Hext et Himsforth en 1962[22], étendues par Nelder et Mead en 1965[17] et revisitées récemment par Dennis et Torczon[4] en 1991.

- Les méthodes de recherche dans une famille de directions

Elles consistent à minimiser successivement la fonction de coût dans une famille de n directions, par une suite d'évaluations. Cette approche a été proposée par Rosenbrock en 1960[21] et modifiée par Powell en 1964[18] et par Swann la même année[24].

- Les méthodes à surface de réponse

Elles consistent à construire un modèle de la fonction de coût, par évaluations de la fonction de coût correspondant à différents vecteurs des variables de contrôle, puis de minimiser ce modèle, ou éventuellement un critère statistique lié au modèle, par une approche classique. Il ne s'agit pas à proprement parler d'une alternative aux méthodes classiques d'optimisation, mais d'une modélisation du problème permettant un recours à d'autres algorithmes d'optimisation, pour un coût de calcul réduit. Ces approches ont notamment été popularisées récemment par Jones[12].

- Les méthodes d'interpolation

Elles consistent à interpoler localement la fonction de coût en différents vecteurs

des variables de contrôle, puis à minimiser localement cette interpolation dans une région de confiance. Le résultat permet alors de mettre à jour les supports de l'interpolation, dans une procédure itérative. Ces méthodes ont été proposées par Winfield en 1973[26], puis revisitées par Powell en 1994[19, 20] et d'autres auteurs récemment[2, 3, 14].

- **Les approches semi-stochastiques :**

- L'algorithme du recuit simulé
Celui-ci s'appuie sur une analogie avec le phénomène de cristallisation en métallurgie. Ce processus alterne des cycles de refroidissement lent et de réchauffage (recuit) qui tendent à minimiser l'énergie du matériau. Cette approche a été proposée par Aarts et Korst en 1989[1].
- Les algorithmes génétiques et évolutionnaires
Les algorithmes génétiques consistent à mimer le processus d'évolution darwinienne d'une population, qui tend à produire des individus les plus adaptés possible à un environnement donné, par modifications de leur génome. Holland a proposé cette approche en 1975[10], qui a été popularisée par Goldberg en 1989[8]. L'approche a été généralisée dans les algorithmes évolutionnaires, basés sur des principes d'évolution plus larges[15].
- Les algorithmes bio-mimétiques (essaims de particules, colonies de fourmis, etc)
Ils sont basés sur une analogie avec le comportement collectif d'organismes vivants. Chaque membre du groupe tend à trouver la meilleure source de nourriture, compte tenu de sa mémoire et des communications venant des autres membres. Ces algorithmes ont été mis en oeuvre par Kennedy et Eberhart (essaims de particules) en 1995[13], ou encore Dorigo[5].

Cette classification n'est pas exhaustive, mais décrit les principales stratégies employées à l'heure actuelle en ingénierie. Dans la suite, on va s'attacher à décrire les principales caractéristiques de quelques algorithmes.

3 Méthodes de descente

Par ces méthodes, le problème (1) est résolu par une approche itérative convergeant vers la solution du problème x^* , les équations d'état étant vérifiées à chaque itération du problème d'optimisation (i.e. les équations d'état sont résolues pour chaque vecteur des variables de contrôle x). A chaque itération k , un nouveau vecteur des variables de contrôle $x^{(k+1)}$ est obtenu à partir *de la valeur de la fonction de coût et de son gradient* et éventuellement d'informations collectées sur le chemin déjà parcouru (voir algorithme (1)).

Algorithm 1 Algorithme modèle pour les méthodes de descente

(0) initialisation

choix d'un vecteur des variables de contrôle initial x^0
 $k \leftarrow 0$

(1) début de la boucle d'optimisation (itération k)

(2) résolution des équations d'état

obtention de $w(x^{(k)})$

(3) calcul de la fonction de coût

obtention de $j(x^{(k)}, w(x^{(k)}))$

(4) calcul du gradient de la fonction de coût

obtention de $\nabla j(x^{(k)}, w(x^{(k)}))$

(5) calcul du nouveau vecteur des variables de contrôle

obtention de $x^{(k+1)}$ à partir de $j(x^{(k)}, w(x^{(k)}))$ et $\nabla j(x^{(k)}, w(x^{(k)}))$

(6) fin de la boucle d'optimisation

$k \leftarrow k + 1$
si convergence atteinte STOP sinon Goto (1)

Dans ce qui suit, on omettra la plupart du temps d'expliciter la dépendance de la fonction de coût vis-à-vis des variables d'état et on notera directement $f(x) = j(x, w(x))$. Il existe de nombreuses méthodes d'optimisation basées sur cet algorithme, correspondant à différentes méthodes de calcul du nouveau vecteur des variables de contrôle à chaque itération (choix de la direction de recherche). On peut par exemple citer la méthode de plus grande descente, pour laquelle :

$$x^{(k+1)} = x^{(k)} - \rho \nabla f(x) \quad (4)$$

ou la méthode de Newton (respectivement quasi-Newton), pour laquelle :

$$x^{(k+1)} = x^{(k)} - \rho \mathcal{H}^{-1}(x) \nabla f(x) \quad (5)$$

où ρ est la longueur de pas dans la direction de recherche choisie et $\mathcal{H}$ la matrice Hessienne de la fonction de coût (respectivement une approximation de la matrice hessienne) agissant comme préconditionneur. Toutes ces méthodes sont largement détaillées dans des ouvrages de référence en optimisation numérique[7, 6]. La méthode la plus utilisée est certainement la méthode de quasi-Newton basée sur le gradient de la fonction coût et une approximation de la matrice Hessienne de type BFGS, mise à jour à chaque itération:

$$\mathcal{H}_{k+1} = \mathcal{H}_k - \frac{1}{s_k^T \mathcal{H}_k s_k} \mathcal{H}_k s_k s_k^T \mathcal{H}_k + \frac{1}{y_k^T s_k} y_k y_k^T \quad (6)$$

$$\text{où } s_k = x^{(k+1)} - x^{(k)} \text{ et } y_k = \nabla f(x^{(k+1)}) - \nabla f(x^{(k)}) \quad (7)$$

Pour compléter la description de la méthode, la longueur de pas ρ est choisie de manière à réaliser effectivement une réduction de la fonction de coût à chaque itération et assurer la convergence de l'algorithme. Les approches classiques reposent sur deux conditions, comme par exemple le critère d'Armijo-Goldstein :

$$f(x^{(k+1)}) < f(x^{(k)}) + \alpha \nabla f(x^{(k)}) \cdot \rho_k d_k \quad (8)$$

$$f(x^{(k+1)}) > f(x^{(k)}) + \beta \nabla f(x^{(k)}) \cdot \rho_k d_k \quad (9)$$

ou le critère d'Armijo-Wolfe :

$$f(x^{(k+1)}) < f(x^{(k)}) + \alpha \nabla f(x^{(k)}) \cdot \rho_k d_k \quad (10)$$

$$\nabla f(x^{(k+1)}) \cdot d_k > \beta \nabla f(x^{(k)}) \cdot d_k \quad (11)$$

où d_k représente la direction de recherche et α, β sont des paramètres numériques. La première condition assure une décroissance de la fonction coût, tandis que le second empêche de réaliser des pas dont la longueur tendrait vers 0.

Ces approches ont été appliquées depuis plusieurs années à des problèmes d'optimisation numérique. Leur emploi a souvent été couronné de succès, ces méthodes disposant de nombreuses qualités : preuve de convergence, taux de convergence superlinéaire possible (faible nombre d'évaluations nécessaire), faible dépendance vis-à-vis de la dimension du problème n , prise en compte de contraintes complexes, ... Néanmoins, plusieurs difficultés limitent l'usage de ces méthodes :

- Pour certains problèmes, la fonction de coût est non-différentiable par nature ou présente des discontinuités (par exemple les problèmes de type Min Max). Dès lors, ces méthode ne peuvent pas être employées sans aménagement spécifique.
- Le calcul du gradient de la fonction de coût peut être délicat. Ce point sera examiné plus précisément plus loin.

- Un bruitage d'erreur, caractérisé par des variations haute fréquence de la fonction de coût, est généralement observé lorsque la fonction de coût est évaluée par des méthodes numériques complexes (code de calcul). Des études ont montré que ce bruitage est amplifié par les méthodes de discrétisation des EDP d'ordres élevés et un critère de convergence faible. Par suite, ce bruitage peut avoir des conséquences funeste pour l'évaluation du gradient de la fonction de coût et mener l'algorithme d'optimisation à l'échec.
- La fonction de coût peut présenter de multiples minima locaux. Or les méthodes classiques ne sont pas capables de détecter et éviter ceux-ci, convergeant vers le premier minimum trouvé, peut-être caractérisé par une valeur de la fonction de coût médiocre, en comparaison du minimum absolu.

Ces différents points expliquent pourquoi les méthodes de descente sont parfois écartées pour la résolution de problèmes complexes à visées industrielles.

4 Calcul du gradient

Pour les problèmes ciblés ici, la fonction coût à minimiser est en fait une *fonctionnelle*, faisant intervenir la solution d'un système d'équations aux dérivées partielles, généralement résolu par une méthode numérique. Par suite, le calcul du gradient de la fonction coût utilisé notamment par les méthodes de descente n'est pas trivial. Différentes approches sont envisageables :

- Approximation par différences finies.
Il s'agit simplement de recourir à plusieurs évaluations pour approcher chaque composante i du gradient selon une formule de différences finies classique :

$$\nabla f(x)|_i = \frac{f(x + \epsilon e_i) - f(x)}{\epsilon} + \mathcal{O}(\epsilon) \quad \nabla f(x)|_i = \frac{f(x + \epsilon e_i) - f(x - \epsilon e_i)}{2\epsilon} + \mathcal{O}(\epsilon^2) \quad (12)$$

Ce choix est généralement à proscrire, étant donné le nombre important d'évaluations à réaliser (au minimum $n+1$). En outre, de larges erreurs peuvent apparaître, dépendant du pas de discrétisation ϵ . Un pas trop large génère une erreur de troncature et un pas trop petit entraîne une erreur d'arrondi. Il est à noter que cette approche est la seule possible lorsque la fonction de coût est évaluée par l'intermédiaire d'un code de calcul commercial, dont les sources sont inconnues.

- Calcul par différentiation automatique[16].
Cette approche, consistant à générer automatiquement un code évaluant la différentielle de la fonction de coût à partir du code source permettant la résolution du système d'EDP, est expéditive mais est encore limitée à des problèmes (et des codes de calcul) modérément complexes.
- Résolution du problème adjoint[11].
Il s'agit de l'approche la plus courante, qui consiste à résoudre un problème linéaire permettant une évaluation aisée du gradient, pour un *coût indépendant du nombre de paramètres*. Deux approches sont possibles, dépendant de l'ordre dans lequel les opérateurs de discrétisation et différentiation sont employés. On présente succinctement ces approches dans la suite.

On présente tout d'abord l'approche *discrète*, pour laquelle on considère les équations d'état sous forme discrétisée. Ainsi, la fonctionnelle coût *discrétisée* s'écrit $J(X, W)$ avec $W = W(X)$ solution des équations d'état *discrètes* :

$$\Psi(X, W) = 0 \quad (13)$$

On applique la règle de dérivation composée :

$$\frac{dJ}{dX_i} = \frac{\partial J}{\partial X_i} + \frac{\partial J}{\partial W} \frac{dW}{dX_i} \quad (14)$$

$$\frac{\partial \Psi}{\partial X_i} + \frac{\partial \Psi}{\partial W} \frac{dW}{dX_i} = 0 \quad (15)$$

Le calcul du gradient de la fonction coût nécessite donc d'évaluer la variation de la solution par rapport aux paramètres de contrôle $\frac{dW}{dX_i}$ (sensibilités). L'approche adjointe vise à éviter le calcul de ce terme, en combinant ces équations et introduisant un multiplicateur de Lagrange Π :

$$\frac{dJ}{dX_i} = \frac{\partial J}{\partial X_i} + \frac{\partial J}{\partial W} \frac{dW}{dX_i} + \Pi^\top \left(\frac{\partial \Psi}{\partial X_i} + \frac{\partial \Psi}{\partial W} \frac{dW}{dX_i} \right) \quad (16)$$

$$\frac{dJ}{dX_i} = \left(\frac{\partial J}{\partial X_i} + \Pi^\top \frac{\partial \Psi}{\partial X_i} \right) + \left(\frac{\partial J}{\partial W} + \Pi^\top \frac{\partial \Psi}{\partial W} \right) \frac{dW}{dX_i} \quad (17)$$

Le terme de sensibilité peut donc être éliminé en choisissant le multiplicateur de Lagrange adéquat. Celui-ci est déterminé en résolvant le système linéaire suivant :

$$\left(\frac{\partial \Psi}{\partial W} \right)^\top \Pi = - \left(\frac{\partial J}{\partial W} \right)^\top \quad (18)$$

où Π sont appelées variables adjointes. Le gradient se déduit alors par :

$$\frac{dJ}{dx} = \frac{\partial J}{\partial X} + \Pi^\top \frac{\partial \Psi}{\partial X} \quad (19)$$

On souligne que le système adjoint est indépendant des variables de contrôle et mesure l'influence de la solution sur la valeur de la fonctions coût.

La seconde approche, qualifiée de *continue*, suit la même idée, mais le raisonnement est réalisé à partir des équations d'état *continues* (équations aux dérivées partielles) $\psi(x, w)$. La fonctionnelle coût *continue* s'écrit :

$$j(x, w) = \int_{\Gamma} \mathcal{M}(x, w) d\Gamma, \quad (20)$$

où Γ représente par exemple la forme de la frontière du domaine (optimisation de forme). La variation totale de j est issue des variations des variables d'état et de contrôle :

$$\delta j = \int_{\Gamma} \frac{\partial \mathcal{M}}{\partial x} \delta x + \frac{\partial \mathcal{M}}{\partial w} \delta w d\Gamma \quad (21)$$

On considère des équations d'état ψ sous forme conservative :

$$\frac{\partial}{\partial \xi} \mathcal{F} + \frac{\partial}{\partial \eta} \mathcal{G} = 0 \quad (22)$$

$$\delta \left(\frac{\partial}{\partial \xi} \mathcal{F} + \frac{\partial}{\partial \eta} \mathcal{G} \right) = \frac{\partial}{\partial \xi} \delta \mathcal{F} + \frac{\partial}{\partial \eta} \delta \mathcal{G} = 0 \quad (23)$$

On ajoute l'intégrale de la variation des équations d'état à celle de la fonctionnelle, en introduisant le champ des variables adjointes Ψ :

$$\delta j = \int_{\Gamma} \frac{\partial \mathcal{M}}{\partial x} \delta x + \frac{\partial \mathcal{M}}{\partial w} \delta w \, d\Gamma + \int_{\Omega} \Psi \left(\frac{\partial}{\partial \xi} \delta \mathcal{F} + \frac{\partial}{\partial \eta} \delta \mathcal{G} \right) d\Omega \quad (24)$$

On intègre par partie le terme volumique :

$$\delta j = \int_{\Gamma} \frac{\partial \mathcal{M}}{\partial x} \delta x + \frac{\partial \mathcal{M}}{\partial w} \delta w + \Psi (\delta \mathcal{F} n_{\xi} + \delta \mathcal{G} n_{\eta}) \, d\Gamma - \int_{\Omega} \left(\frac{\partial \Psi}{\partial \xi} \delta \mathcal{F} + \frac{\partial \Psi}{\partial \eta} \delta \mathcal{G} \right) d\Omega \quad (25)$$

La variation totale du flux s'écrit :

$$\delta \mathcal{F} = \underbrace{\frac{\partial \mathcal{F}}{\partial x} \delta x}_{\text{seulement sur } \Gamma} + \frac{\partial \mathcal{F}}{\partial w} \delta w \quad \delta \mathcal{G} = \underbrace{\frac{\partial \mathcal{G}}{\partial x} \delta x}_{\text{seulement sur } \Gamma} + \frac{\partial \mathcal{G}}{\partial w} \delta w \quad (26)$$

On isole les différentes contributions :

$$\begin{aligned} \delta j = & \int_{\Gamma} \left(\frac{\partial \mathcal{M}}{\partial x} + \Psi \left(\frac{\partial \mathcal{F}}{\partial x} n_{\xi} + \frac{\partial \mathcal{G}}{\partial x} n_{\eta} \right) \right) \delta x + \left(\frac{\partial \mathcal{M}}{\partial w} + \Psi \left(\frac{\partial \mathcal{F}}{\partial w} n_{\xi} + \frac{\partial \mathcal{G}}{\partial w} n_{\eta} \right) \right) \delta w \, d\Gamma \\ & - \int_{\Omega} \left(\frac{\partial \Psi}{\partial \xi} \frac{\partial \mathcal{F}}{\partial w} + \frac{\partial \Psi}{\partial \eta} \frac{\partial \mathcal{G}}{\partial w} \right) \delta w \, d\Omega \end{aligned} \quad (27)$$

Finalement, on annule la partie dépendant des sensibilités δw en imposant les variables adjointe de manière adéquat. Pour cela, on définit l'équation adjointe sous forme d'EDP :

$$\begin{aligned} \frac{\partial \Psi}{\partial \xi} \frac{\partial \mathcal{F}}{\partial w} + \frac{\partial \Psi}{\partial \eta} \frac{\partial \mathcal{G}}{\partial w} &= 0 \quad (\Omega) \\ \frac{\partial \mathcal{M}}{\partial w} + \Psi \left(\frac{\partial \mathcal{F}}{\partial w} n_{\xi} + \frac{\partial \mathcal{G}}{\partial w} n_{\eta} \right) &= 0 \quad (\Gamma) \end{aligned} \quad (28)$$

La variation totale de la fonctionnelle se calcule alors par :

$$\delta j = \int_{\Gamma} \left(\frac{\partial \mathcal{M}}{\partial x} + \Psi \left(\frac{\partial \mathcal{F}}{\partial x} n_{\xi} + \frac{\partial \mathcal{G}}{\partial x} n_{\eta} \right) \right) \delta x \, d\Gamma \quad (29)$$

L'équation adjointe doit alors être discrétisée, souvent de manière analogue à l'équation d'état, pour être résolue numériquement. Comme pour l'approche discrète, l'équation adjointe ne fait pas apparaître les paramètres de contrôle.

Les deux approches présentées ci-dessus fournissent des estimations du gradient différentes, mais convergentes lorsque le pas de discrétisation tend vers 0, car les opérateurs de discrétisation et différentiation ne sont pas commutatifs. On retiendra que l'approche discrète permet d'être consistant avec la solution numérique des équations d'état, tandis que l'approche continue évite de dériver certains termes introduits lors de la résolution numérique.

5 Prise en compte des contraintes

5.1 Conditions d'optimalité

5.1.1 Cas non contraint

On considère tout d'abord dans cette partie un problème non linéaire sans contrainte, de dimension n :

$$\text{minimiser } f(x) \quad x \in \mathbb{R}^n \quad (30)$$

Deux conditions nécessaires pour que x^* soit une solution du problème sont :

$$\begin{aligned} g(x^*) &= 0 \\ H(x^*) &\text{ semi-définie positive} \end{aligned} \quad (31)$$

Pour démontrer ces conditions, on fait appel à un développement de Taylor-Lagrange à proximité de x^* , dans la direction p :

$$f(x^* + \epsilon p) = f(x^*) + \epsilon p^T g(x^*) + \frac{1}{2} \epsilon^2 p^T H(x^* + \epsilon \theta p) p \quad (32)$$

avec $0 \leq \theta \leq 1$. La première condition se démontre par contradiction. Si $g(x^*)$ est non nul, il existe p tel que $p^T g(x^*) < 0$. On peut alors trouver une quantité ϵ telle que $f(x^* + \epsilon p) < f(x^*)$, ce qui est en contradiction avec la caractérisation du minimum. Par suite, le développement de Taylor au deuxième ordre se limite à :

$$f(x^* + \epsilon p) = f(x^*) + \frac{1}{2} \epsilon^2 p^T H(x^* + \epsilon \theta p) p \quad (33)$$

La seconde condition est également démontrée par contradiction. Si $H(x^*)$ est indéfinie, alors H est indéfinie dans un voisinage de x^* . On peut alors trouver une quantité ϵ et une direction p tel que $\epsilon^2 p^T H(x^* + \epsilon \theta p) p < 0$, conduisant à l'existence d'un point en contradiction avec la caractérisation du minimum. Ces deux conditions ne sont cependant pas suffisantes. On montre que les conditions suffisantes pour que x^* soit une solution du problème sont :

$$\begin{aligned} g(x^*) &= 0 \\ H(x^*) &\text{ définie positive} \end{aligned}$$

(34)

Si le gradient s'annule en x^* , on peut utiliser un développement limité de la forme 33. Si $H(x^*)$ est définie positive, alors H est définie positive dans un voisinage de x^* . Par suite, pour tout ϵ suffisamment petit et tout p , on a $\epsilon^2 p^T H(x^* + \epsilon \theta p) p > 0$ et $f(x^* + \epsilon p) > f(x^*)$, donc x^* est une solution d'après la caractérisation d'un minimum local.

5.1.2 Contraintes linéaires

On cherche dans ce paragraphe à déterminer des conditions d'optimalité pour un problème incluant des contraintes linéaires d'égalité. Un tel problème peut s'écrire :

$$\begin{aligned} &\text{minimiser } f(x) \quad x \in \mathbb{R}^n \\ &\text{soumis à } \widehat{A}x = \widehat{b} \end{aligned} \quad (35)$$

où $\hat{A}$ est la matrice de taille $m \times n$ des contraintes, dont la i -ème ligne est notée $\hat{a}_i^T$. D'après la caractérisation d'un minimum, un point admissible x^* est une solution, si $f(x^*) < f(x)$ pour tout x admissible dans un voisinage de x^* . Afin de déterminer les conditions d'optimalité, on cherche à caractériser les points admissibles du problème.

Il n'existe pas de points admissibles si les contraintes ne sont pas compatibles. On supposera donc par la suite que les contraintes sont linéairement indépendantes. Une caractérisation des points admissibles peut être simplement déterminée en utilisant la propriété de linéarité des contraintes. Si on considère un déplacement entre deux points admissibles x et y , on a par linéarité $\hat{A}(x - y) = \hat{A}(x) - \hat{A}(y) = 0$. On en déduit que tout déplacement p entre deux points admissibles vérifie :

$$\hat{A}p = 0 \quad (36)$$

ce qui est équivalent à p orthogonal aux vecteurs-lignes de $\hat{A}$. Un vecteur p vérifiant la condition 36 est appelé *direction admissible*. Tout déplacement à partir d'un point admissible selon une direction admissible génère un point qui respecte les contraintes, puisque $\hat{A}(x + \alpha p) = \hat{A}x + \alpha \hat{A}p = \hat{A}x = \hat{b}$.

On considère désormais une base du sous espace de $\mathbb{R}^n$ dont les vecteurs vérifient le critère 36. On note Z la matrice dont les vecteur-colonnes sont ces vecteurs de base. Toute direction admissible p peut s'écrire comme une combinaison linéaire des vecteur-colonnes de Z , ce qui peut s'exprimer sous la forme :

$$p = Zp_z \quad (37)$$

En utilisant la relation 36, on obtient :

$$\hat{A}Z = 0 \quad (38)$$

Afin de déterminer les conditions d'optimalité du problème, on fait appel à un développement de Taylor-Lagrange au voisinage d'un point admissible x^* dans une direction admissible $p = Zp_z$:

$$f(x^* + \epsilon Zp_z) = f(x^*) + \epsilon p_z^T Z^T g(x^*) + \frac{1}{2} \epsilon^2 p_z^T Z^T H(x^* + \epsilon \theta p) Zp_z \quad (39)$$

On utilise ensuite un raisonnement analogue à celui utilisé précédemment pour le cas sans contrainte. Pour que x^* soit une solution du problème, on peut montrer qu'il est nécessaire que le terme $p_z^T Z^T g(x^*)$ s'annule pour tout p_z , ce qui conduit à la condition nécessaire :

$$Z^T g(x^*) = 0 \quad (40)$$

Une conséquence de 38 et 40 est que le gradient en x^* peut s'exprimer comme une combinaison linéaire des vecteurs lignes de $\hat{A}$:

$$g(x^*) = \sum_{i=1}^m \hat{a}_i \lambda_i^* = \hat{A}^T \lambda^* \quad (41)$$

Le vecteur λ^* est le vecteur des *multiplicateurs de Lagrange*. En utilisant des arguments analogues au cas sans précédent, on peut montrer que la semi-définie positivité du terme

$Z^T H(x^*)Z$ est une autre condition nécessaire. Finalement, on a établi trois conditions nécessaires pour que x^* soit une solution du problème 35 : x^* doit vérifier les contraintes, le gradient projeté doit être nul en x^* et la matrice hessienne projetée doit être semi-définie positive en x^* :

$$\begin{aligned}\widehat{A}x^* &= \widehat{b} \\ Z^T g(x^*) &= 0 \text{ ou } g(x^*) = \widehat{A}^T \lambda^* \\ Z^T H(x^*)Z &\text{ semi-définie positive}\end{aligned}\tag{42}$$

Inversement, il est aisé de montrer que les conditions suivantes sont suffisantes, en s'inspirant du cas sans contrainte :

$$\begin{aligned}\widehat{A}x^* &= \widehat{b} \\ Z^T g(x^*) &= 0 \text{ ou } g(x^*) = \widehat{A}^T \lambda^* \\ Z^T H(x^*)Z &\text{ définie positive}\end{aligned}$$

(43)

5.1.3 Contraintes non linéaires

On considère dans ce paragraphe un problème faisant intervenir des contraintes non-linéaires d'égalité.

$$\begin{aligned}\text{minimiser} \quad & f(x) & x &\in \mathbb{R}^n \\ \text{soumis à} \quad & \widehat{c}_i(x) = 0 & i &= 1, \dots, t\end{aligned}\tag{44}$$

La détermination des conditions d'optimalité associées à un tel problème fait intervenir des raisonnements proches de ceux évoqués pour un problème avec contraintes linéaires. On note ainsi $\widehat{A}(x)$ la matrice dont les vecteurs-lignes représentent le gradient des contraintes au point x et $Z(x)$ la matrice dont les vecteurs-colonnes forment une base de l'espace orthogonal au sous espace engendré par les vecteurs-lignes de $\widehat{A}(x)$. On se contentera dans ce paragraphe d'énoncer les conditions d'optimalité, dans la mesure où leur démonstration est plus complexe que dans le cas linéaire. Contrairement au cas linéaire, la courbure des contraintes intervient dans les conditions d'optimalité du problème. On définit le Lagrangien de la fonctionnelle f :

$$L(x, \lambda) = f(x) - \lambda^T \widehat{c}(x)\tag{45}$$

Pour tenir compte de la courbure des contraintes, la condition d'optimalité du second ordre fait intervenir le hessien du Lagrangien W à la place du hessien de la fonctionnelle. Plus précisément, les conditions nécessaires associées au problème 44 sont :

$$\begin{aligned}\widehat{c}(x) &= 0 \\ Z^T g(x^*) &= 0 \text{ ou } g(x^*) = \widehat{A}^T \lambda^* \\ Z^T W(x^*, \lambda^*)Z &\text{ semi-définie positive}\end{aligned}\tag{46}$$

Les conditions suffisantes se résument à :

$$\boxed{\begin{aligned} \widehat{c}(x) &= 0 \\ Z^T g(x^*) &= 0 \text{ ou } g(x^*) = \widehat{A}^T \lambda^* \\ Z^T W(x^*, \lambda^*) Z &\text{ définie positive} \end{aligned}} \quad (47)$$

Le lecteur désireux d'approfondir ce point pourra trouver une analyse détaillée dans [6, 7].

5.2 Fonctions de pénalisation

L'utilisation de fonctions de pénalisation est un moyen simple pour transformer un problème incluant des contraintes quelconques en un problème sans contrainte, indépendamment de l'algorithme de résolution utilisé. On suppose qu'on souhaite minimiser la fonctionnelle f , soumise à des contraintes non-linéaires. On définit la fonction pénalisée quadratique :

$$f_q(x, \rho) = f(x) + \rho \widehat{c}(x)^T \widehat{c}(x) \quad (48)$$

où $\widehat{c}(x)$ représente les contraintes violées au point x . Le terme ajouté à la fonctionnelle pénalise ainsi les points non-admissibles. ρ est appelé *coefficient de pénalité*. La fonctionnelle f_q obtenue est continûment différentiable. On peut montrer avec quelques hypothèses classiques [7] que, en notant x^* le minimum de f et $x^*(\rho)$ le minimum de f_q selon x , on a :

$$\lim_{\rho \rightarrow \infty} x^*(\rho) = x^* \quad (49)$$

En conséquence, on peut obtenir la solution du problème initial en résolvant une séquence de sous-problèmes sans contrainte, pour lesquels on cherche à minimiser f_q selon x , avec une valeur de ρ croissante. Cette approche a cependant plusieurs inconvénients. Tout d'abord, de nombreux sous-problèmes doivent être résolus, occasionnant un coût important en terme de calculs. Ensuite, on peut montrer [7] que l'accroissement de ρ conduit à des sous-problèmes mal posés, qui créent des difficultés pour les algorithmes de résolution.

Pour surmonter ces difficultés, on peut avoir recours à une fonction de pénalité en valeur absolue. Celle-ci se définit par :

$$f_a(x, \rho) = f(x) + \rho \|\widehat{c}(x)\|_1 \quad (50)$$

La fonctionnelle f_a n'est pas dérivable aux points où une contrainte s'annule. En revanche, elle conduit à un problème mieux posé que la fonction de pénalisation quadratique f_q . En effet, on peut montrer [7] qu'il existe une valeur de seuil $\bar{\rho}$ pour le coefficient de pénalité, telle que :

$$x^*(\rho) = x^* \quad \forall \rho > \bar{\rho} \quad (51)$$

Pour obtenir la solution du problème initial avec contrainte, il suffit donc de résoudre quelques sous-problèmes sans contrainte consistant à minimiser f_a , en augmentant si nécessaire la valeur de ρ jusqu'à ce que les contraintes soient vérifiées. Cette approche simple à mettre

en oeuvre est couramment utilisée en pratique, car elle permet d'appréhender aisément tous les types de contraintes. Néanmoins, son application soulève une difficulté majeure : le choix du coefficient de pénalité initial. Si celui-ci est trop faible, la résolution du sous-problème conduit à une solution qui viole largement les contraintes. S'il est trop fort, on tend comme pour le cas précédent vers un sous-problème mal posé. En pratique, quelques essais préalables doivent donc être réalisés de manière à déterminer un bon ordre de grandeur pour le coefficient de pénalité.

5.3 Fonctions de barrière

L'emploi de fonctions de pénalité permet de déterminer une solution admissible à un problème avec contraintes, mais génère aux cours des itérations des points non-admissibles. En conséquence, cette approche n'est pas acceptable lorsque l'admissibilité doit être strictement préservée. Ce cas se produit notamment lorsqu'on souhaite définir des limites au domaine de variation des variables du problème. Les fonctions de barrière sont conçues pour prendre en compte ce type de contraintes. Elles sont comparables aux fonctions de pénalité, dans la mesure où elles consistent à construire un sous-problème en ajoutant un terme de pénalisation à la fonctionnelle. En revanche, ce terme pénalise les points admissibles, de manière croissante au fur et à mesure qu'on se rapproche des contraintes. Par suite, les contraintes ne peuvent être violées et cette approche est limitée aux contraintes d'inégalité. On considère la minimisation de la fonctionnelle f , soumise à la contrainte $c(x) > 0$. La fonction de barrière logarithmique est définie par :

$$f_b(x, r) = f(x) - r \ln(c(x)) \quad (52)$$

En notant $x^*(r)$ le minimum de f_b selon x , on peut montrer similairement aux fonctions de pénalité [7] que :

$$\lim_{r \rightarrow 0} x^*(r) = x^* \quad (53)$$

La résolution d'une séquence de sous-problèmes sans contrainte, pour lesquels on minimise f_b , avec une valeur décroissante de r , permet donc d'obtenir la solution du problème initial en assurant l'admissibilité des points parcourus. Cette approche souffre malheureusement des mêmes maux que les fonctions de pénalité quadratiques : de nombreux sous-problèmes doivent être résolus et on tend progressivement vers des sous-problèmes mal posés.

Rosenbrock [21] propose une approche pragmatique, utilisée pour fixer des limites au domaine de variation des variables. Une pénalisation locale de la fonctionnelle est réalisée, de manière permanente. Ainsi, on résout un seul sous-problème, dont la solution est une approximation de la solution du problème initial. Pour un problème de maximisation, Rosenbrock propose de multiplier f par une fonctionnelle, qui prend pour valeur 0 aux limites du domaine $(l_i^{inf})_{i=1,\dots,n}$ et $(l_i^{sup})_{i=1,\dots,n}$, puis croît quadratiquement vers 1 sur une courte distance ϵ . Cette méthode est cependant limitée aux problèmes de maximisation pour lesquels la fonctionnelle f prend des valeurs positives. En s'inspirant de cette approche, on peut

définir une fonction de barrière logarithmique locale, pour chaque direction i , de la forme :

$$\begin{aligned}\Phi_i(x) &= 0 \quad \text{si} \quad l_i^{inf} + \epsilon < x_i < l_i^{sup} - \epsilon \\ &= \frac{x_i - (l_i^{inf} + \epsilon)}{\epsilon} - \ln\left(\frac{x_i - l_i^{inf}}{\epsilon}\right) \quad \text{si} \quad x_i < l_i^{inf} + \epsilon \\ &= \frac{(l_i^{sup} - \epsilon) - x_i}{\epsilon} - \ln\left(\frac{l_i^{sup} - x_i}{\epsilon}\right) \quad \text{si} \quad l_i^{sup} + \epsilon < x_i\end{aligned}\tag{54}$$

On considère alors le sous-problème consistant à minimiser :

$$f_b(x) = f(x) + \sum_{i=1}^n \Phi_i(x)\tag{55}$$

Cette fonctionnelle est continûment dérivable. La solution du sous-problème est une approximation de la solution du problème initial, dépendant du paramètre ϵ , lorsque la solution se situe à proximité d'une limite du domaine. L'introduction de cette technique est motivée par la nécessité de s'assurer en pratique du caractère borné du domaine de variation des variables.

5.4 Méthode de projection du gradient

Cette approche, initialement développée pour un problème à contraintes linéaires, est dérivée des conditions d'optimalité correspondantes 43. On note $\hat{A}_k$ la matrice des t contraintes linéaires actives à l'itération courante k et Z_k la matrice dont les vecteurs-colonnes forment une base du sous-espace orthogonal au sous-espace généré par les vecteur-lignes de $\hat{A}_k$. Par suite, toute direction de recherche p_k s'écrivant $Z_k p_z$, avec p_z vecteur de dimension $n - t$, est une direction admissible qui ne viole pas les contraintes. Un algorithme modèle peut donc être construit en générant une suite de points admissibles, en prenant pour p_z une direction de descente. Par exemple, on peut considérer la projection du gradient de la fonctionnelle, ce qui donne un algorithme du type méthode de plus grande descente :

$$p_z = -Z_k^T g_k\tag{56}$$

Un algorithme de type quasi-Newton peut également être construit en considérant la projection d'une approximation du hessien de la fonctionnelle. La direction de recherche est alors déterminée par résolution du système :

$$Z_k^T \tilde{H}_k Z_k p_z = -Z_k^T g_k\tag{57}$$

Cette approche peut être étendue aux problèmes possédant des contraintes non-linéaires, en considérant le gradient des contraintes au point courant. Cependant, les directions de recherche définies précédemment ne génèrent plus des points admissibles, car elles ne tiennent pas compte de la courbure des contraintes. En conséquence, une correction doit être effectuée

pour maintenir l'admissibilité. En pratique, le déplacement réalisé à chaque itération prend la forme :

$$s_k = \alpha_k p_k + Y_k p_y, \quad (58)$$

où les colonnes de Y_k forment une base du sous-espace généré par les vecteur-lignes de $\hat{A}_k$. Cette correction peut être délicate à contrôler lorsque les contraintes sont fortement non-linéaires. Des détails sont fournis dans [7].

5.5 Méthode du Lagrangien projeté

L'examen des conditions d'optimalité pour un problème incluant des contraintes non-linéaires 47 pousse à faire intervenir le Lagrangien de la fonctionnelle dans l'algorithme de résolution. En notant $\hat{c}(x)$ les contraintes actives au point x , le Lagrangien de la fonctionnelle est défini par :

$$L(x, \lambda) = f(x) - \lambda^T \hat{c}(x) \quad (59)$$

En utilisant les conditions d'optimalité 47, on montre aisément que la solution du problème x^* est un minimum selon x du Lagrangien dans le sous-espace orthogonal au gradient des contraintes actives en (x^*, λ^*) . En effet, x^* est un point stationnaire du Lagrangien selon x d'après 41 et le hessien du Lagrangien projeté est défini positif en (x^*, λ^*) . x^* peut donc être perçu comme la solution d'un problème à contraintes linéaires faisant intervenir le Lagrangien. En pratique, le couple solution (x^*, λ^*) est inconnu. Les algorithmes consistent donc à résoudre une séquence de sous-problèmes, en considérant comme contraintes linéaires le gradient des contraintes et une approximation des multiplicateurs de Lagrange au point courant. L'implémentation pratique de cette approche peut être complexe [7], les méthodes résultantes sont toutefois considérées comme étant très performantes.

6 Méthode de type simplexe

La méthode “Multi-directional Search Algorithm” (MSA) développée par Torczon[25], fait partie des méthodes de type simplexe et a été conçue pour une utilisation sur architecture parallèle.

Cette méthode consiste à déplacer un simplexe de $n + 1$ sommets dans $\mathbb{R}^n$ (un triangle dans $\mathbb{R}^2$, un tétraèdre dans $\mathbb{R}^3$, ...), chaque sommet représentant un vecteur de n variables de contrôle. L’algorithme débute par une initialisation, consistant à générer l’ensemble des sommets du simplexe $s_0^{(0)}, s_1^{(0)}, \dots, s_n^{(0)}$, par exemple en perturbant successivement d’une quantité δ les composantes d’un vecteur initial des variables de contrôle $s_0^{(0)} = x^{(0)}$:

$$s_i^{(0)} = s_0^{(0)} + \delta e_i \quad \text{avec } e_i^T = [\delta_{1i}, \dots, \delta_{ni}] \quad i = 1, \dots, n \quad (60)$$

où δ_{ji} est le symbole de Kronecker. Ensuite, les déplacements du simplexe sont réalisés de manière à faire décroître la fonction de coût évaluée au meilleur des sommets. A chaque itération k , on suppose que $s_0^{(k)}$ est le meilleur sommet du simplexe. L’ensemble des sommets du simplexe est déplacé homothétiquement par rapport à $s_0^{(k)}$:

$$\widetilde{s}_i^{(k)} = \alpha s_i^{(k)} + (1 - \alpha) s_0^{(k)} \quad i = 0, \dots, n \quad (61)$$

avec α généralement pris égal à -1. Ce déplacement est appelé *réflexion*. En cas de succès, i.e. au moins un des sommets obtient une valeur de la fonction de coût inférieure à celle de $s_0^{(k)}$, le simplexe subit alors une *expansion*, destinée à poursuivre la recherche dans les directions considérées :

$$s_i^{(k+1)} = \gamma \widetilde{s}_i^{(k)} + (1 - \gamma) \widetilde{s}_0^{(k)} \quad i = 0, \dots, n \quad (62)$$

avec γ généralement pris égal à 2. Au contraire, en cas d’échec lors de la réflexion, le simplexe subit une *contraction*, pour laquelle les sommets se rapprochent du meilleur sommet $s_0^{(k)}$:

$$s_i^{(k+1)} = \beta \widetilde{s}_i^{(k)} + (1 - \beta) \widetilde{s}_0^{(k)} \quad i = 0, \dots, n \quad (63)$$

avec β généralement pris égal à 1/2. Après avoir réalisé une expansion ou une contraction, l’algorithme débute une nouvelle itération. Une vue synthétique de la méthode est donnée par l’algorithme (2).

Le méthode MSA effectue donc une recherche du vecteur des variables de contrôle optimales par une suite de n évaluations simultanées et indépendantes de la fonction de coût. Ces instanciations peuvent être réalisées sur une architecture parallèle comprenant n processeurs. L’algorithme résultant effectue donc $2n$ évaluations de la fonction de coût par itération, pour un coût en terme de temps de calcul de 2 évaluations de la fonction de coût par itération.

La méthode possède une preuve de convergence. On suppose que $\{s_0^{(k)}\}$ est la suite des meilleurs sommets du simplexe. On définit le domaine :

$$L(s_0^{(0)}) = \{x \in \mathbb{R}^n : f(x) \leq f(s_0^{(0)})\} \quad (64)$$

(0) initialisation

initialisation par perturbation $s_i^{(0)} = s_0^{(0)} + \delta e_i \quad i = 1, \dots, n$
évaluation de la fonction de coût en $s_i^{(0)} \quad i = 0, \dots, n$
 $k \leftarrow 0$

(1) début de la boucle d'optimisation (itération k)**(2) mouvement de réflexion**

recherche du meilleur sommet $s_0^{(k)}$
réflexion : $\widetilde{s}_i^{(k)} = \alpha s_i^{(k)} + (1 - \alpha) s_0^{(k)} \quad i = 0, \dots, n$
évaluation de la fonction de coût en $\widetilde{s}_i^{(k)} \quad i = 1, \dots, n$

(3) si amélioration : mouvement d'expansion

expansion : $s_i^{(k+1)} = \gamma \widetilde{s}_i^{(k)} + (1 - \gamma) s_0^{(k)} \quad i = 0, \dots, n$
évaluation de la fonction de coût en $s_i^{(k+1)} \quad i = 1, \dots, n$

(4) sinon : mouvement de contraction

contraction : $s_i^{(k+1)} = \beta \widetilde{s}_i^{(k)} + (1 - \beta) s_0^{(k)} \quad i = 0, \dots, n$
évaluation de la fonction de coût en $s_i^{(k+1)} \quad i = 1, \dots, n$

(6) fin de la boucle d'optimisation

$k \leftarrow k + 1$
si convergence atteinte STOP sinon Goto (1)

Pour $y \in \mathbb{R}^n$, on définit le contour :

$$C(y) = \{x \in \mathbb{R}^n : f(x) = f(y)\} \quad (65)$$

On note X^* l'ensemble des points stationnaires de f dans $L(s_0^{(0)})$. On peut ainsi montrer[4] le théorème suivant : Si $L(s_0^{(0)})$ est un compact et f est continûment différentiable sur $L(s_0^{(0)})$, alors il existe une sous-suite de $\{s_0^{(k)}\}$ qui converge vers $x^* \in X^*$. Par suite, $\{s_0^{(k)}\}$ converge vers $C^* = C(x^*)$ dans le sens où :

$$\lim_{k \rightarrow \infty} \left[\inf_{x \in C^*} \|s_0^{(k)} - x\| \right] = 0 \quad (66)$$

Ce résultat peut être obtenu à partir de deux points essentiels : les vecteurs $s_0^{(k)} - s_i^{(k)}$ sont linéairement indépendants par construction. Par suite, au moins un de ces vecteurs

n'est pas orthogonal à $\nabla f(s_0^{(k)})$ et produira une direction de descente si $s_0^{(k)}$ n'est pas un point stationnaire. L'algorithme ne peut donc pas converger vers un point non stationnaire. En outre, une suite d'itérations avec échecs génère pour chaque direction une recherche unidimensionnelle de type "backtracking", ce qui assure le résultat de convergence précédent.

Cette approche peut finalement se présenter comme une alternative aux méthodes de descente. Elle permet de s'affranchir du calcul du gradient de la fonction de coût. Par suite, elle est bien adaptée aux problèmes pour lesquels la fonction de coût présente un fort bruitage d'erreur. En revanche, elle converge *a priori* vers un optimum local et n'assure pas de déterminer l'optimum global de la fonction de coût.

7 Algorithmes génétiques

Les algorithmes génétiques s'appuient sur un paradigme darwinien de l'évolution génétique d'une population. Dans la nature, les individus d'une population se trouvent en compétition, par exemple lors de la recherche de nourriture ou pour la reproduction. D'après les théories de l'évolution, les individus les plus adaptés survivent majoritairement et ont une descendance. Par suite, les caractéristiques génétiques de ces individus sont transmis à la génération suivante, qui voit potentiellement sa performance augmenter. Le patrimoine génétique de la population s'améliore ainsi de générations en générations.

Un modèle caricatural de cette évolution est mise en oeuvre par les algorithmes génétiques : un procédé de codage est introduit, de manière à générer une structure génétique à partir des variables du problème d'optimisation. On définit ensuite des opérateurs permettant l'évolution du patrimoine génétique de la population d'une génération à l'autre. Ces opérateurs font largement appel à des lois aléatoires. La performance d'un individu, qui détermine sa capacité à survivre, est liée à la valeur de la fonction de coût. On peut définir la performance comme l'inverse ou l'opposé de la fonction de coût. La procédure d'optimisation consiste alors à simuler l'évolution d'une population, durant un certain nombre de générations, jusqu'à la détermination d'un individu optimal.

La mise en oeuvre de ces algorithmes a montré leur robustesse et leur capacité à déterminer le minimum global d'un problème. Cependant, le nombre d'évaluations de la fonctionnelle à effectuer est considérablement supérieur au nombre d'évaluations requis par un algorithme classique.

Il existe de nombreux algorithmes génétiques, qui sont généralement des variantes de l'algorithme canonique développé par Holland[10]. L'algorithme débute par le choix d'un codage permettant de représenter les variables du problème d'optimisation par une structure génétique. Une population initiale est ensuite créée en générant n_{ind} individus aléatoirement. La performance de ces individus est alors calculée, nécessitant n_{ind} évaluations de la fonction de coût.

A chaque génération, trois opérateurs interviennent, modifiant les caractéristiques génétiques de la population. L'opérateur de *sélection* agit en premier. Son rôle est de choisir les individus de la population qui vont survivre. Le critère déterminant la sélection est lié à la performance des individus. La population après sélection compte toujours n_{ind} individus, certains individus de la population ayant disparu et d'autres ayant été dupliqués. L'opérateur de *croisement* intervient ensuite. Il crée de nouveaux individus en recombinant le patrimoine génétique de la population. Pour cela, les individus sont appariés aléatoirement, chaque couple de parents créant deux enfants par un échange partiel de chromosomes. A la fin de l'étape de croisement, la population compte n_{ind} individus enfants. Enfin, l'opérateur de *mutation* crée de nouveaux individus en introduisant de nouvelles caractéristiques génétiques dans la population des n_{ind} enfants. Certains chromosomes sont perturbés aléatoirement, de manière à explorer l'espace de recherche. L'algorithme se poursuit alors avec une nouvelle génération.

Cet algorithme canonique est illustré par le schéma algorithmique 3 et les différentes étapes sont décrites dans les paragraphes suivants.

Algorithm 3 Algorithme génétique canonique

- (0) initialisation**
 - choix d'un codage
 - génération aléatoire de n_{ind} individus
 - $k \leftarrow 1$
 - (1) évaluation de la population (génération k)**
 - calcul de la performance des n_{ind} individus
 - (2) sélection**
 - détermination de n_{ind} survivants
 - (3) croisement**
 - création de n_{ind} enfants à partir des survivants
 - (4) mutation**
 - modification de certains enfants
 - (5) mise a jour**
 - $k \leftarrow k + 1$ Goto (1)
-

Les algorithmes génétiques diffèrent des autres méthodes d'optimisation car ils utilisent un codage des variables de contrôle, plutôt que les variables de contrôle elles-mêmes. Le codage désigne le processus qui transforme les variables de contrôle en un *chromosome*.

$$x = [(x)_1, (x)_2, \dots, (x)_n] \in \mathbb{R}^n \rightarrow [a_1, a_2, \dots, a_l] \quad (67)$$

Les éléments constituant le chromosome sont les *gènes*. Ils appartiennent à un ensemble appelé *alphabet*. L'algorithme canonique emploie un codage binaire qui s'appuie sur l'alphabet $\{0, 1\}$, pour des raisons qui apparaîtront plus tard.

Le codage binaire suppose en pratique la définition d'un intervalle admissible pour les variables de contrôle et une discrétisation de cet intervalle. Pour chaque variable de contrôle réelle $(x)_i$, on définit une limite inférieure l_i^{inf} et une limite supérieure l_i^{sup} au domaine de variation. En supposant que la discrétisation fournisse une précision de N_i^{cs} chiffres significatifs, le nombre de gènes nécessaire au codage de la variable $(x)_i$ est le plus petit entier l_i vérifiant :

$$(l_i^{sup} - l_i^{inf})10^{N_i^{cs}} \leq 2^{l_i} - 1 \quad (68)$$

Un chromosome étant constitué des gènes associés à chaque variable de contrôle mis bout à bout, le nombre total de gènes d'un chromosome, caractérisant un individu et un point dans

l'espace de recherche est alors :

$$l = \sum_{i=1}^n l_i \quad (69)$$

La transformation d'un chromosome en variables réelles peut se faire de manière naturelle par la relation :

$$(x)_i = l_i^{inf} + \frac{l_i^{sup} - l_i^{inf}}{2^{l_i} - 1} \sum_{i_{bit}=1}^{l_i} 2^{i_{bit}-1} Ch(i_{bit}) \quad (70)$$

où $Ch(i_{bit})$ représente la valeur binaire, 0 ou 1, du gène i_{bit} associé à la variable $(x)_i$. On considère pour illustration un exemple simple, comprenant deux variables de contrôle. Les données sont les suivantes :

$$\begin{aligned} l_1^{inf} &= 0. & l_1^{sup} &= 7. & N_1^{cs} &= 0 \\ l_2^{inf} &= -2. & l_2^{sup} &= 13. & N_2^{cs} &= 0 \end{aligned} \quad (71)$$

Grâce à la relation 68, on détermine le nombre de gènes nécessaires au codage de chaque variable.

$$l_1 = 3 \quad l_2 = 4 \quad (72)$$

Un chromosome est donc caractérisé par $l = 7$ gènes. On considère, par exemple, le chromosome [1001101] et on cherche à connaître les variables réelles qui lui sont associées, en appliquant la relation 70. La première variable est caractérisée par le début de chaîne [1001101] et correspond à la valeur

$$(x)_1 = 0. + 1.(2^0 \times \mathbf{1} + 2^1 \times \mathbf{0} + 2^2 \times \mathbf{0}) = 1. \quad (73)$$

La seconde variable est caractérisée par la fin de la chaîne [1001101] et a pour valeur

$$(x)_2 = -2. + 1.(2^0 \times \mathbf{1} + 2^1 \times \mathbf{1} + 2^2 \times \mathbf{0} + 2^3 \times \mathbf{1}) = 9. \quad (74)$$

Après avoir défini un codage, une population initiale de n_{ind} individus est générée aléatoirement. Chaque individu est caractérisé par un chromosome comptant l gènes et représente un point de l'espace de recherche. L'initialisation de la population peut donc être réalisée simplement en effectuant $n_{ind} \times l$ tirages au sort binaires, de manière à générer n_{ind} chromosomes. Le tirage aléatoire doit avoir une loi de probabilité uniforme, de façon à obtenir une occupation homogène de l'espace de recherche.

A chaque génération, on évalue la *performance* de chaque individu de la population, ce qui nécessite n_{ind} évaluations de la fonction de coût. Pour un problème de minimisation, la valeur opposée ou inverse de la fonction de coût peut être choisie. Ensuite, des opérateurs font évoluer les caractéristiques génétiques de la population, de manière à explorer l'espace de recherche et favoriser les caractéristiques des meilleurs individus pour les générations futures.

Le rôle de l'opérateur de sélection est de filtrer la population de manière à conserver les individus possédant de "bonnes" caractéristiques génétiques. La seule mesure dont on dispose de la qualité d'un individu est sa performance, liée à la valeur de la fonction de coût. Il semble donc naturel de sélectionner les individus pour lesquels la valeur de la fonction de coût est la meilleure et d'éliminer les autres. Néanmoins, cette approche doit être appréhendée avec précaution, puisqu'on souhaite conserver pour les générations suivantes les meilleures caractéristiques génétiques et pas seulement les meilleurs individus. Un individu peut en effet avoir certains gènes présents dans la solution optimale mais une performance médiocre, et son élimination serait néfaste. Cependant, les individus ayant une bonne performance sont plus susceptibles de posséder les bons gènes. On prend ainsi conscience de la notion de *pression de sélection*. Si la pression est trop forte, seul un petit nombre d'individus excellents survivent et la convergence de l'algorithme risque d'être prématurée, conduisant à un optimum local. Si la pression est trop faible, peu d'individus médiocres sont éliminés à chaque génération et la progression de la population est lente. En pratique, les opérateurs doivent tenir compte de cette notion de manière équilibrée.

l'algorithme canonique emploie un opérateur appelé *sélection par roulette*. Celle-ci simule le fonctionnement d'une roulette de jeux pour déterminer les individus à sélectionner. A chaque individu correspond une portion de la roue et un lancer de roue est simulé par un tirage aléatoire. On sélectionne alors l'individu dont la portion de roue est désignée. On effectue n_{ind} lancers, de manière à sélectionner n_{ind} individus. Par conséquence, certains individus disparaissent de la population, tandis que d'autres sont dupliqués. Pour que la sélection soit efficace, les individus possédant la meilleure performance doivent être préférentiellement sélectionnés. On utilise donc une roulette "truquée", la portion allouée à chaque individu étant choisie proportionnellement à sa performance. La probabilité p_i pour un individu i de performance f_i d'être sélectionné est :

$$p_i = \frac{f_i}{\sum_{j=1}^{n_{ind}} f_j} \quad (75)$$

Un exemple de sélection par roulette pour quatre individus est fourni par le tableau 1.

L'opérateur de croisement génère de nouveaux individus à partir des chromosomes des individus sélectionnés. Un échange partiel des chromosomes entre individus est réalisé, produisant donc de nouveaux individus, correspondant à de nouveaux points dans l'espace de recherche. Cependant, aucun gène nouveau n'est introduit dans la population. Par suite, cet opérateur effectue seulement une redistribution des gènes. Les individus sélectionnés possédant certaines caractéristiques génétiques intéressantes, on tente de recombinaison celles-ci à travers l'opérateur de croisement.

Lors de cette étape, les individus sont groupés par paire de parents de manière aléatoire. On réalise ensuite un échange partiel des chromosomes avec une probabilité p_c , généralement proche de l'unité. Une forte probabilité favorise l'échange d'informations, mais augmente le risque de perdre de bonnes combinaisons.

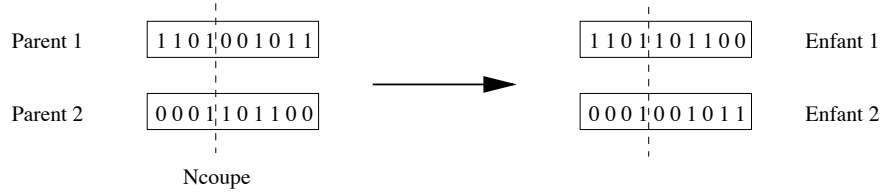
L'algorithme canonique emploie une technique appelée *coupure à un point*. Il consiste à choisir aléatoirement un point de coupe sur les deux chromosomes parents, puis à échanger

individu	performance	probabilité	portion
1	0.565	0.18	$0 \rightarrow 0.18$
2	0.628	0.20	$0.18 \rightarrow 0.38$
3	0.377	0.12	$0.38 \rightarrow 0.50$
4	1.571	0.50	$0.50 \rightarrow 1.00$

No tirage	résultat aléatoire	individu sélectionné
1	0.354	2
2	0.879	4
3	0.567	4
4	0.157	1

Table 1: Exemple de sélection par roulette

les gènes à partir de ce point de coupe pour obtenir deux nouveaux individus enfants, selon le schéma ci-dessous :



Enfin, le rôle de la mutation est d'introduire de nouvelles caractéristiques génétiques, ou de les réintroduire, en modifiant quelques gènes des individus enfants. Pour chaque gène de chaque enfant, une permutation est réalisée avec une probabilité p_m . Cette valeur est généralement faible, de manière à ne pas transformer l'algorithme en une simple recherche aléatoire. La mutation permet d'explorer l'espace de recherche aléatoirement et de réintroduire certains gènes qui ont disparu au cours des générations.

8 Stratégies d'évolution

Les stratégies d'évolution peuvent être considérées comme une généralisation des algorithmes génétiques. Ces derniers ont en effet plusieurs inconvénients, du moins pour l'algorithme canonique présenté ci-dessus : discrétisation des variables continues, nombreux paramètres numériques à régler, difficulté d'appréhender le rôle des différents opérateurs.

Les stratégies d'évolution se basent sur seulement deux opérateurs, la mutation et la sélection, et utilisent les variables continues du problème d'optimisation. L'algorithme canonique [15] repose sur une population générant un ensemble de λ enfants par mutation. Cette

dernière est suivie d'une étape de sélection aboutissant à μ parents. Si cette sélection est effectuée parmi les λ enfants seuls, l'algorithme est noté (μ, λ) . Si elle repose sur l'ensemble des λ enfants et μ parents, la méthode est notée $(\mu + \lambda)$. Les parents sont ensuite utilisés pour mettre à jour les caractéristiques globales de la population pour la génération suivante. A chaque génération k , la population est caractérisée par sa moyenne $\bar{x}_k$ et sa variance $\bar{\sigma}_k^2$.

Une stratégie d'évolution typique est décrite par le schéma algorithmique 4.

Algorithm 4 Stratégie d'évolution de type (μ, λ) ou $(\mu + \lambda)$

(0) initialisation

choix de $\bar{x}_0$ et $\bar{\sigma}_0^2$

$k \leftarrow 0$

(1) mutation

création de λ enfants à partir de $\bar{x}_k$ et $\bar{\sigma}_k^2$ (génération k)

(2) évaluation de la population

calcul de la performance des λ enfants

(3) sélection

détermination de μ survivants

(4) recombinaison

mise à jour des caractéristiques de la population à partir des survivants

(5) mise a jour

$k \leftarrow k + 1$ Goto (1)

L'opérateur de mutation se définit à partir de ces caractéristiques statistiques. Plus précisément, les λ enfants sont générés par perturbation aléatoire de la moyenne, selon une loi normale:

$$x_i = \bar{x}_k + \sigma_i N(0, Id). \quad (76)$$

σ_i est l'écart type de la perturbation, propre à chaque individu. Il évolue selon une loi d'apprentissage:

$$\sigma_i = \bar{\sigma}_k e^{\tau N(0,1)}, \quad (77)$$

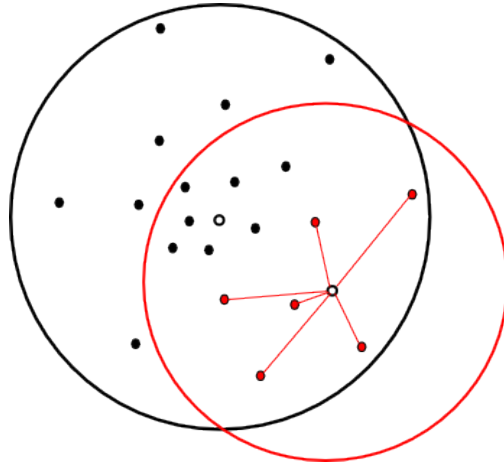
où τ est un paramètre numérique fixé.

L'opérateur de sélection consiste simplement à retenir les meilleurs μ individus parmi les λ enfants générés par mutation (pour la version (μ, λ)). Finalement, les caractéristiques

statistiques de la population sont mises à jour à partir des caractéristiques des survivants :

$$\bar{x}_{k+1} = \frac{1}{\mu} \sum_{i=1}^{\mu} x_i \quad \bar{\sigma}_{k+1} = \frac{1}{\mu} \sum_{i=1}^{\mu} \sigma_i. \quad (78)$$

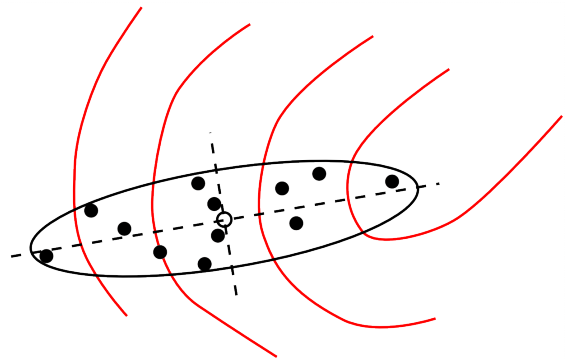
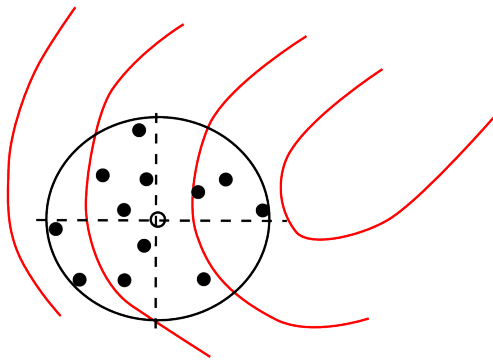
On souligne le rôle de l'évolution de la variance au long de l'optimisation, qui donne à l'algorithme un caractère *auto-adaptatif* et se révèle important pour l'efficacité de la méthode. L'approche est illustrée par la figure suivante, où les points représentent les individus enfants d'une génération donnée et les points rouges les survivants permettant la mise à jour du barycentre pour la génération suivante.



On constate que ces algorithmes sont plus simples à mettre en oeuvre qu'un algorithme génétique, notamment grâce à l'absence de codage, et reposent sur peu de paramètres numériques à régler. Il existe des preuves de convergence vers l'optimum global, sous des hypothèses classiques. Néanmoins, ces preuves sont de nature statistique. Il est par exemple possible de montrer que :

$$\forall \epsilon > 0 \quad \lim_{k \rightarrow \infty} P(|f(x_k) - f(x^*)| \leq \epsilon) = 1. \quad (79)$$

Enfin, il faut souligner que le coût de ces algorithmes, en terme de nombre d'évaluations à réaliser, est beaucoup plus important que pour les approches déterministes. Par suite, leur emploi est souvent limité à un nombre de paramètres faible (quelques dizaines), même si de nouvelles approches, telles l'algorithme CMA-ES (Covariance Matrix Adaption)[9] représentent une réelle amélioration de performance en introduisant notamment la notion d'anisotropie de la distribution. Ceci est illustré sur la figure suivante, montrant à gauche une distribution isotrope d'un algorithme ES standard et à droite une distribution anisotrope adaptée à la fonction à optimiser (algorithme CMA-ES).



9 Optimisation multicritère

Les algorithmes génétiques et les stratégies d'évolution vus précédemment sont mieux adaptés à la résolution de certains problèmes qu'un simple optimiseur déterministe. L'optimisation multicritère est un exemple typique pour lequel ces algorithmes apportent une méthodologie originale et innovante.

Tout au long des paragraphes précédents, on cherchait à atteindre un point optimal selon un critère unique. Cependant, dans le domaine des sciences de l'ingénieur, l'optimalité est rarement définie par un seul critère. Ainsi, il existe généralement un ensemble d'indications, devant être intégrées lors des prises de décision. Le but de cette section est de décrire quelques approches destinées à traiter un problème multicritère, et souligner l'apport des algorithmes évolutionnaires dans ce domaine.

9.1 Définitions

Problème Un problème d'optimisation multicritère consiste à optimiser simultanément plusieurs fonctions de coût, représentant chacune un critère de décision. Formellement, le problème peut s'écrire :

$$\text{minimiser } f(x) = (f_1(x), \dots, f_p(x)) \quad x \in \mathbb{R}^n \quad (80)$$

Les fonctions de coût $f_1(x), \dots, f_p(x)$ ne possèdent généralement pas un minimum commun. En conséquence, on ne peut généralement pas déterminer une solution unique à un problème multicritère. La solution est composée d'un ensemble de points représentant des compromis entre les différents critères.

Optimalité au sens de Pareto De manière à préciser la nature des points solution, on est amené à définir le concept de *dominance*. Un point x domine un point y si x est aussi performant que y pour tous les critères et est strictement meilleur pour au moins un critère :

$$x_c^1 \text{ domine } x_c^2 \Leftrightarrow \exists i \, f_i(x_c^1) < f_i(x_c^2) \text{ et } f_j(x_c^1) \leq f_j(x_c^2) \, \forall j \neq i \quad (81)$$

Lorsqu'on résout un problème d'optimisation multicritère, on cherche à obtenir *l'ensemble des points non-dominés*, appelé *front de Pareto*. En effet, ces points ne sont pas comparables entre eux, dans la mesure où il n'est pas possible de déterminer dans cet ensemble un point dominant un autre. Ils représentent donc des compromis entre les différents critères. Pour illustrer cette notion, on considère l'exemple à deux critères représenté par la figure (1). Le point x_c^1 est dominé par x_c^5 et domine x_c^4 . En revanche, il n'est ni meilleur ni pire que $(x_c^2$ et x_c^3 . Finalement, l'espace objectif peut être réparti en trois zones : la zone en gris foncé correspond aux points qui dominent x_c^1 , celle en gris clair aux points dominés par x_c^1 , la zone blanche représentant les points indifférents. Un exemple de front de Pareto est donné par la figure (2).

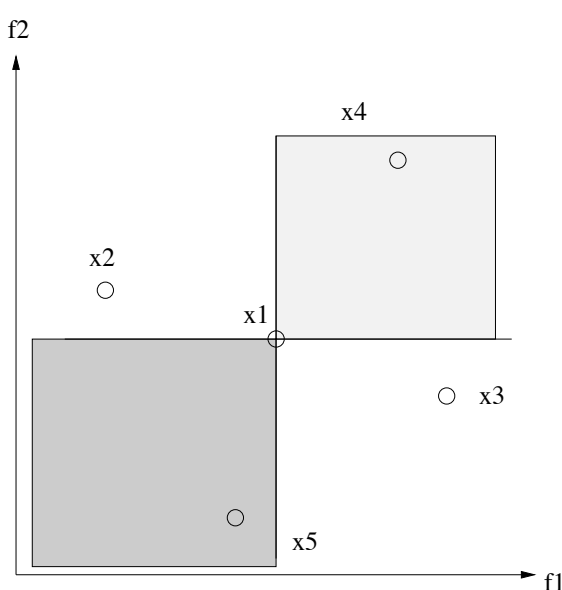


Figure 1: Notion de dominance

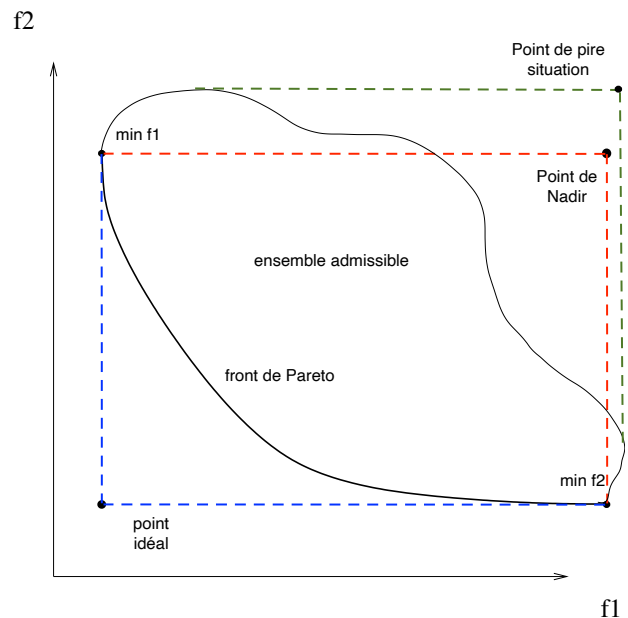


Figure 2: Front de Pareto

9.2 Méthode des pondérations

Les algorithmes étudiés dans les chapitres précédents ne peuvent prendre en compte qu'une unique fonction de coût. Pour traiter un problème multicritère, l'idée vient naturellement de construire une fonction de coût composite, représentative des différents critères, puis de la minimiser à l'aide d'un des algorithmes déjà décrits. Une approche classique consiste à introduire une pondération des différents critères :

$$\text{minimiser } f_{pond} = \sum_{i=1}^p \omega_i f_i \quad \sum_{i=1}^p \omega_i = 1 \quad (82)$$

Le choix des pondérations détermine l'importance relative des critères, chaque optimisation permettant de déterminer un point du front de Pareto. Outre le fait que de nombreux calculs doivent être réalisés pour obtenir une représentation du front, cette méthode a un inconvénient majeur : certains points du front de Pareto ne peuvent pas être atteints lorsque le front n'est pas convexe. Pour mettre en évidence ce phénomène, on considère le cas à deux paramètres exposé à la figure (3), pour lequel on cherche à minimiser la fonction de coût $f = \omega_1 f_1 + \omega_2 f_2$. En écrivant $f_2 = -\frac{\omega_1}{\omega_2} f_1 + \frac{1}{\omega_2} f$, on constate que le point atteint par la minimisation se situe au point de tangence du front et de la droite de pente $s = -\frac{\omega_1}{\omega_2}$, dont l'ordonnée à l'origine des abscisses est la plus faible possible. Par conséquent, pour toute

pondération appliquée, les points du front sur la figure (3) entre x_c^2 et x_c^3 ne peuvent être obtenus par cette méthode. Lorsque $s < s_2$, les points atteints se situent à gauche de x_c^2 et lorsque $s > s_2$, ils sont à droite de x_c^3 . les points de la partie concave du front ne peuvent pas être atteints. Cette approche se révèle finalement très limitée dans son champ d’application.

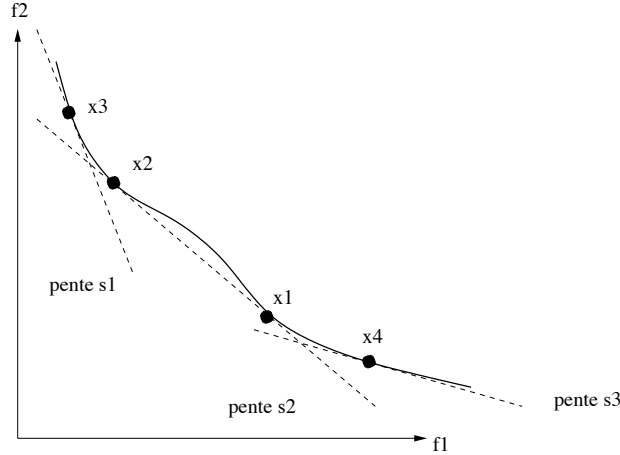


Figure 3: Méthode des pondérations

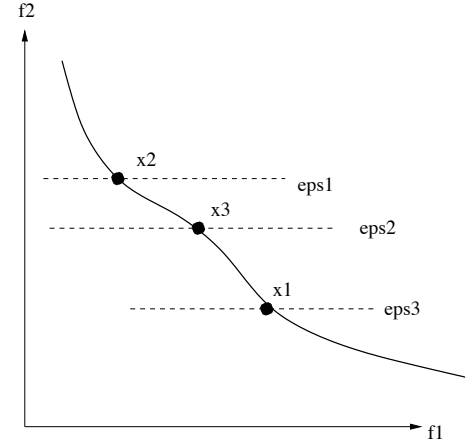


Figure 4: Méthode des contraintes

9.3 Méthode des contraintes

Une formulation alternative consiste à considérer $p - 1$ fonctions comme des contraintes et se ramener ainsi à une optimisation mono-objectif soumise à $p - 1$ contraintes additionnelles :

$$\text{minimiser } f_1 \quad \text{soumis à} \quad f_i(x) = \epsilon_i \quad i = 2, \dots, p \quad (83)$$

avec ϵ_i un ensemble de valeurs fixées par l’utilisateur et devant varier pour parcourir le front de Pareto. Le principal inconvénient de cette approche est le grand nombre de problèmes sous contraintes à résoudre pour obtenir une approximation satisfaisante du front. On souligne que, contrairement au cas précédent, le caractère concave du front ne pose pas de problème particulier ici.

9.4 Approches génétiques

Contrairement aux algorithmes déterministes limités à la méthode des pondérations, les algorithmes génétiques se prêtent tout particulièrement à l’optimisation multicritère, dans la mesure où la solution du problème est composée d’un ensemble de points. Différentes stratégies ont été développées, pour lesquelles l’opérateur de sélection est modifié afin de tendre vers une population répartie le long du front de Pareto. On décrit dans ce cours l’approche “Non-Dominated Sorting Genetic Algorithm” (NSGA)[23].

Cette méthode propose d’attribuer aux individus une performance biaisée, dépendant de la distance qui les sépare du front de Pareto. A chaque génération, avant sélection, la

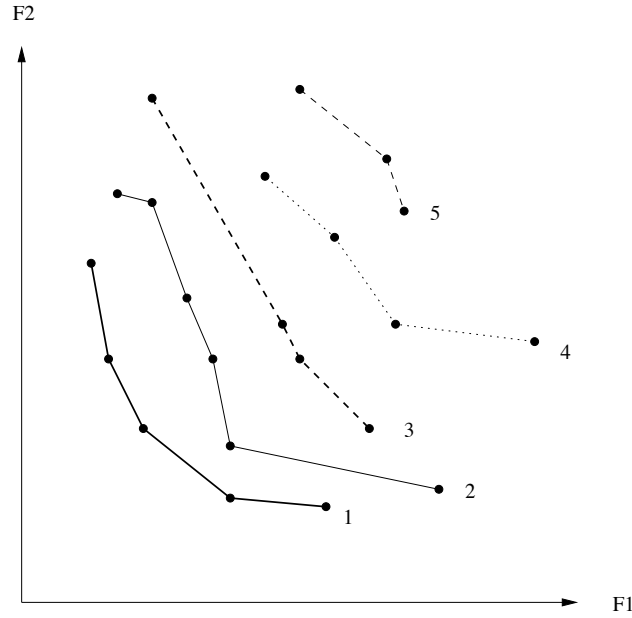


Figure 5: Classement par fronts

population est classée par *fronts*. Les individus non-dominés constituent le front 1. Parmi les individus restants, les non-dominés forment le front 2, et ainsi de suite ... Un exemple est donné par la figure (5). Ensuite, une performance biaisée est attribuée à chaque individu, dépendant du front auquel il appartient. Les individus constituant le premier front se voient attribuer une même performance élevée. Ensuite, une stratégie de partage entre les individus du front est réalisée, suivant la distance qui les sépare dans l'espace phénotypique. Cette mesure favorise la diversité et permet l'étalement de la population le long du front de Pareto. Les individus du second front sont ensuite traités de la même manière, avec une performance initiale inférieure à la plus basse performance attribuée dans le front précédent. La procédure se poursuit ainsi jusqu'à ce qu'une performance biaisée soit associée à chaque individu de la population. La sélection s'opère ensuite par un processus classique, en considérant comme critère la performance biaisée. Cette méthode est particulièrement simple à intégrer à un algorithme unicritère, puisqu'il suffit d'insérer la procédure d'attribution de la performance biaisée.

10 Optimisation robuste

10.1 Notion d'incertitudes

L'optimisation considérée jusqu'à présent s'appuie sur l'hypothèse que l'évaluation de la fonction de coût est parfaite, autrement dit que le processus de simulation du phénomène physique sous jacent (mécanique, thermique, électromagnétique, ...) décrit exactement la réalité. Cette hypothèse est bien entendu fautive et, par suite, un tel cadre correspond à une optimisation idéalisée. Les conséquences peuvent être importantes, dans la mesure où un système optimisé pour des conditions idéales peut voir sa performance sérieusement dégradée dans des conditions réelles.

Généralement, on distingue deux sources d'écart entre la valeur estimée et réelle de la fonction coût : les *erreurs* et les *incertitudes*. Les erreurs désignent tous les défauts identifiés et réductibles, comme par exemple la discrétisation des EDP, la présence de critères d'arrêt, la simplification de la géométrie, la précision arithmétique des calculs, etc. Les incertitudes désignent tous les défauts dus à un manque de connaissances du problème ne pouvant pas être réduits *a priori*, comme par exemple les conditions exactes de fonctionnement (météo), les états de surface des matériaux (tolérances de fabrication), les coefficients intervenant dans les modèles, etc. On souligne que cette distinction peut parfois être floue et varier selon les domaines scientifiques considérés. On retiendra l'idée que les erreurs sont généralement contrôlées lors de la simulation du système à optimiser, tandis que les incertitudes sont modélisées en introduisant des *variables aléatoires* dans le système. On suppose qu'on est capable de caractériser ces variables aléatoires, par exemple à travers la connaissance de leur densité de probabilité.

L'objectif de l'optimisation robuste est de prendre en compte les incertitudes lors de l'optimisation. Par souci de simplicité, on suppose dans ce qui suit qu'on a une seule source d'incertitude modélisée par la variable aléatoire ξ . Celle-ci peut impacter les variables d'optimisation, i.e. $x(\xi)$, ou n'importe quelle étape de la simulation décrivant l'état, i.e. $w(x, \xi)$, ou encore l'évaluation de la fonction coût, i.e. $j(x, w(x), \xi)$. On peut bien entendu cumuler ces effets dans certains cas et le problème de conception optimale devient donc formellement :

$$\begin{aligned} &\text{Minimiser } j(x(\xi), w(x(\xi), \xi), \xi) \\ &\text{Soumis à } \psi(x(\xi), w(x(\xi), \xi), \xi) = 0 \end{aligned} \tag{84}$$

Pour simplifier la description des méthodes, on implique encore la dépendance de la fonction de coût vis à vis des variables d'état et on suppose donc qu'on cherche à minimiser la fonction $f(x(\xi), \xi)$, où x est le vecteur des paramètres d'optimisation et ξ une variable aléatoire de densité de probabilité ρ_ξ . On considère également une contrainte d'égalité, impactée également par l'incertitude, qui s'écrit $c(x(\xi), \xi) = 0$.

10.2 Examen du problème

À ce stade, il faut examiner différentes approches possibles :

- **Approches probabilistes** : Celles-ci s'appuient sur des mesures statistiques de la performance. On distingue deux grandes familles de méthodes :

- **Optimisation robuste**

On parle d'optimisation robuste lorsqu'on s'intéresse aux *fluctuations fréquentes* de la variable incertaine, qui impactent la performance du système sans que cela soit critique pour son fonctionnement (p. ex. la fluctuation quotidienne du vent pour un avion). L'approche standard consiste alors à minimiser l'*espérance* de la fonction de coût μ_j (pour obtenir la meilleure performance en moyenne), et éventuellement de minimiser sa *variance* σ_j^2 (pour réduire les fluctuations de performance). On cherche à satisfaire la contrainte en moyenne. On aboutit donc à un problème à deux objectifs, qui peut se formuler :

$$\begin{aligned} \text{Minimiser } \begin{cases} \mu_j(x) = \int_{\Omega_\xi} j(x(\xi), \xi) \rho_\xi d\xi \\ \sigma_j^2(x) = \int_{\Omega_\xi} (j(x(\xi), \xi) - \mu_j)^2 \rho_\xi d\xi \end{cases} & \quad (85) \\ \text{Soumis à } \mu_c(x) = \int_{\Omega_\xi} c(x(\xi), \xi) \rho_\xi d\xi = 0. & \end{aligned}$$

- **Optimisation fiabiliste**

On parle d'optimisation fiabiliste lorsqu'on s'intéresse à une *fluctuation extrême*, qui impacte possiblement l'intégrité du système (p. ex. une tempête décennale pour un avion). Dans ce cas, on cherche plutôt à s'assurer de la fiabilité du système en exigeant que la probabilité de défaillance soit très faible :

$$P[|c(x(\xi), \xi)| > \epsilon] < p, \quad (86)$$

où ϵ représente la limite de défaillance et p la probabilité de risque.

- **Approches déterministes** : Contrairement aux cas précédents, ces méthodes ne s'appuient pas sur des mesures statistiques. On peut citer les approches suivantes :

- **Optimisation du pire cas**

Cette approche consiste à améliorer la performance pour les conditions les pires possibles. Si on suppose que le paramètre incertain varie dans un intervalle $[\xi_{min}; \xi_{max}]$, on identifie pour chaque évaluation de la fonction coût la valeur correspondant au pire cas :

$$j_{max}(x) = \max\{j(x(\xi), \xi); \xi \in [\xi_{min}; \xi_{max}]\} \quad (87)$$

Le problème d'optimisation consiste à minimiser cette grandeur :

$$\text{Minimiser } j_{max}(x). \quad (88)$$

Globalement, on a donc affaire à un problème de type min-max, comprenant deux problèmes d'optimisation imbriqués. On souligne que la minimisation de j_{max} est *a priori* non différentiable. Cette méthode est finalement proche dans son objectif de l'optimisation fiabiliste.

– **Optimisation par intervalles**

Comme pour la méthode précédente, on identifie pour chaque évaluation de la fonction coût la valeur correspondant au pire cas et au meilleur cas :

$$\begin{aligned} j_{max}(x) &= \max\{j(x(\xi), \xi); \xi \in [\xi_{min}; \xi_{max}]\} \\ j_{min}(x) &= \min\{j(x(\xi), \xi); \xi \in [\xi_{min}; \xi_{max}]\} \end{aligned} \quad (89)$$

Le problème d'optimisation à deux objectifs consiste à minimiser la valeur médiane de la performance et l'intervalle de performance :

$$\text{Minimiser } \begin{cases} \frac{1}{2} (j_{min}(x) + j_{max}(x)) \\ \frac{1}{2} (j_{max}(x) - j_{min}(x)) \end{cases} \quad (90)$$

Cette méthode se rapproche de l'optimisation robuste, car elle vise également à réduire les fluctuations de performance.

Pour ces deux méthodes, la contrainte peut être évaluée au point correspondant au cas le pire, ou éventuellement prise en compte par sa valeur médiane.

10.3 Résolution des sous-problèmes

Pour toutes les approches citées précédemment, on doit résoudre un problème d'optimisation principal (par exemple minimiser la moyenne et variance de f) dont le critère dépend du résultat d'un sous-problème (quantifier la moyenne et la variance). Il en résulte une difficulté algorithmique plus importante et surtout un effort de calcul fortement accru.

10.3.1 Estimation des grandeurs statistiques

L'approche standard pour évaluer l'espérance et la variance de la fonction coût, pour un point x donné, consiste à utiliser une méthode de *Monte-Carlo* et un estimateur statistique associé à un échantillon de taille N :

$$\mu_j(x) \approx \frac{1}{N} \sum_{i=1}^N f(x(\xi_i), \xi_i) \quad \sigma_j^2(x) \approx \frac{1}{N-1} \sum_{i=1}^N (f(x(\xi_i), \xi_i) - \mu_j(x))^2. \quad (91)$$

Cette approche multiplie le coût de calcul d'une optimisation par la taille de l'échantillon, ce qui peut rapidement devenir prohibitif. Une approche alternative consiste à construire un *méta-modèle* $\xi \rightarrow \hat{f}_x(\xi) \approx f(x(\xi), \xi)$, i.e. modèle de modèle, à partir d'un échantillon de taille faible et permettant une estimation rapide des statistiques. La probabilité de satisfaction de la contrainte peut être traitée de la même manière.

10.3.2 Estimation des valeurs extrêmes

Estimer les valeurs minimales et maximales de la fonction coût est encore plus coûteux, dans la mesure où il s'agit de résoudre un problème d'optimisation pour x donné. Comme pour l'estimation des grandeurs statistiques, on rencontre fréquemment deux approches, consistant respectivement à déterminer les valeurs minimales et maximales parmi un échantillon de taille N , ou à construire un méta-modèle plus facile à étudier.

A Problème d'optimisation structurale

A.1 Exposé du problème

On illustre les différents algorithmes vus dans ce cours par un problème d'optimisation structurale d'un portique. L'objectif est d'optimiser un système composé de deux barres à profil cylindrique, soumis à un chargement. Le système correspond au schéma suivant :

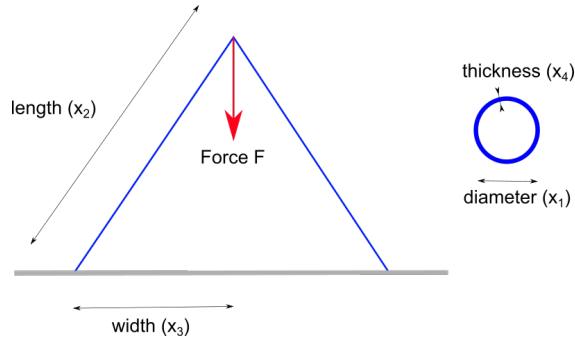


Figure 6: vue générale à gauche, vue d'une section à droite.

Le problème d'optimisation peut être formulé de la manière suivante :

$$\begin{aligned} &\text{Minimiser} \quad vol(x_1, x_2, x_3, x_4) \\ &\text{soumis à} \quad \sigma(x_1, x_2, x_3, x_4) < 400 \end{aligned}$$

où vol représente le volume de la structure et σ les contraintes mécaniques internes à la structure. Ainsi, on cherche à minimiser la quantité de matière utilisée pour la fabrication. La contrainte de $400 N.cm^{-2}$ correspond à l'effort mécanique interne admissible. La force appliquée est supposée valoir en moyenne $F = 150000 N$. Les variables d'optimisation peuvent varier dans les intervalles suivants (en cm):

$$20 < x_1 < 80 \quad 800 < x_2 < 1200 \quad 700 < x_3 < 800 \quad 2 < x_4 < 3$$

Le volume se calcule de manière évidente par:

$$vol = 2\pi x_1 x_2 x_4 10^{-6}$$

L'effort mécanique interne est donné par :

$$\sigma = \frac{x_2 F}{2\pi x_1 x_4 \sqrt{x_2^2 - x_3^2}}$$

A.2 comparaison des algorithmes

On résout ce problème en utilisant une méthode de descente (quasi-Newton BFGS), une méthode de type simplexe (algorithme de Torczon), une stratégie d'évolution (algorithme (μ, λ)). Etant donné les différences d'échelle des variables, on utilise une adimensionnalisation de manière à se ramener à des variables d'optimisation variant entre -1 et +1. On choisit comme point de départ le centre du domaine de recherche et un pas initial de valeur 0.2 pour tous les algorithmes. La contrainte est prise en compte par une pénalisation absolue, avec un coefficient fixe de valeur 100. La stratégie d'évolution utilise une population de taille 8.

La figure suivante permet de comparer la performance des algorithmes. On constate

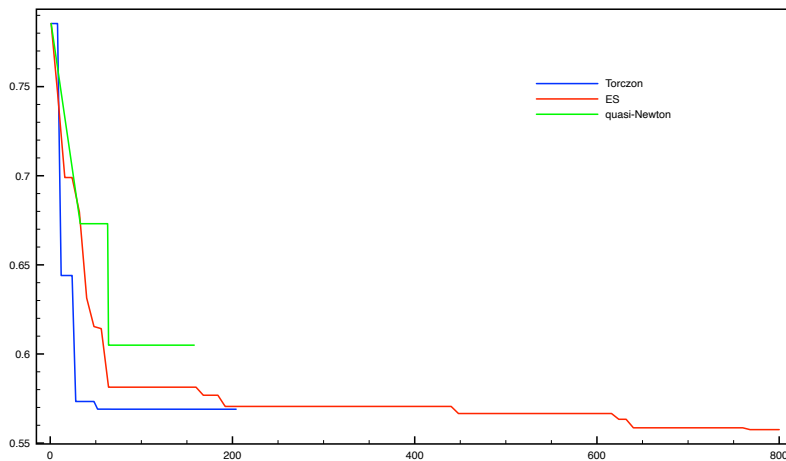


Figure 7: Evolution du volume en fonction du nombre de simulations.

que la méthode de descente converge rapidement vers un minimum local, la méthode de Torczon est légèrement plus performante. La stratégie d'évolution aboutit à la meilleure performance, mais pour un nombre de simulations considérablement supérieur.

Il est intéressant de tester l'influence du choix du point de départ. Pour cela, on utilise l'algorithme de Torczon et on choisit 4 autres points de départ aléatoirement. On obtient les résultats présentés à la figure 8. Dans tous les cas, une valeur assez satisfaisante est trouvée, mais pour chaque cas le volume final est différent. En observant l'évolution de la première variable, on constate que l'algorithme a convergé vers des points différents, selon le point de départ initial. Le problème est donc clairement multimodal et possède plusieurs minima locaux.

Dans le cas de la stratégie d'évolution, il est intéressant d'observer l'impact de la génération des nombres aléatoires. Pour cela, on réalise 4 essais avec les mêmes paramètres en changeant uniquement l'initialisation du générateur aléatoire. Le résultat est donné par la figure 9. Comme on le constate sur la figure du haut (population de taille 8), l'impact est

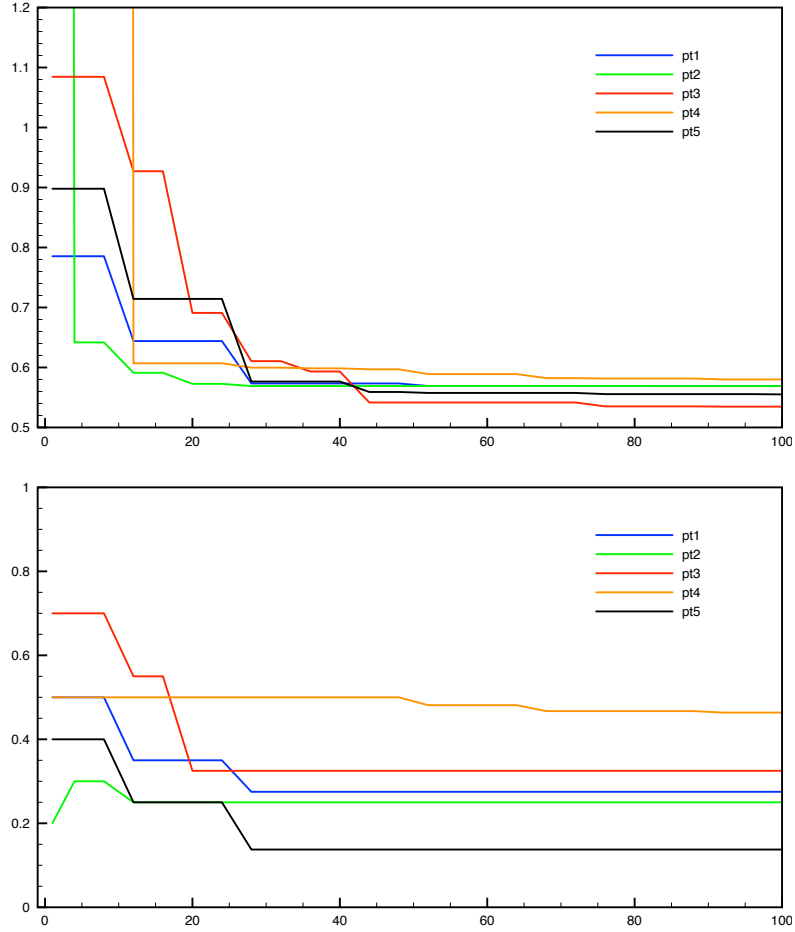


Figure 8: Etude de l'influence du point de départ : évolution du volume (haut) et de la variable x_1 (bas) en fonction du nombre de simulations.

faible, car tous les essais aboutissent à une performance satisfaisante, mais la valeur finale est cependant différente. C'est un point faible de ce type d'algorithme. Si on veut réduire cet effet stochastique dans la recherche, on peut augmenter la taille de la population, comme illustré sur la figure du bas (population de taille 200), ce qui permet d'obtenir une meilleure performance avec moins d'aléat, mais pour un coût de calcul plus important.

A.3 Problème multi-objectif

Dans un second temps, on traite ce problème avec deux objectifs, le volume et l'effort mécanique interne, dans le but de déterminer le front de Pareto. Pour cela, on ré-utilise dans un premier temps un algorithme mono-objectif pour minimiser le volume en faisant varier la valeur de la contrainte. Dans un second temps, on effectue une vraie résolution multi-objective à l'aide d'un algorithme évolutionnaire. La figure 10 compare les résultats obtenus

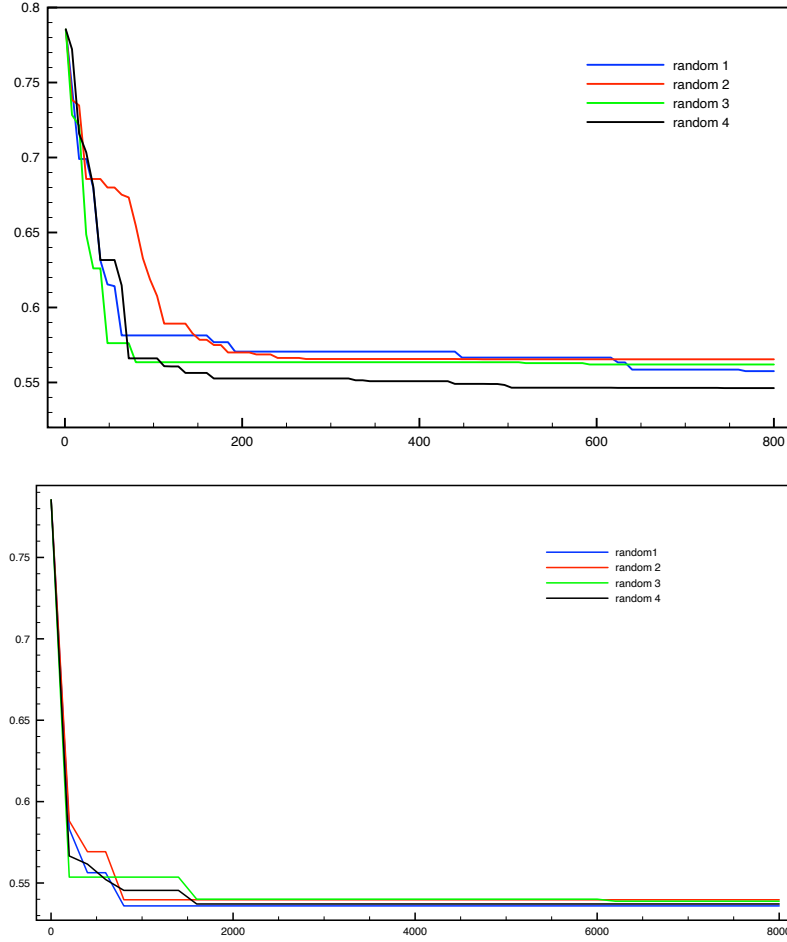


Figure 9: Etude de l'influence du générateur aléatoire : évolution du volume en fonction du nombre de simulations (haut : population 8 ; bas : population 200).

en choisissant 5 valeurs de contrainte et un algorithme évolutionnaire sur 100 générations. On constate que la méthode évolutionnaire permet en un seul essai d'obtenir une bonne approximation du front, même si la distribution des points le long du front n'est pas uniforme.

A.4 Problème avec incertitude

Finalement, on suppose que le chargement F est une variable aléatoire, de distribution normale avec une moyenne de valeur 150 000 et un écart-type de valeur 15 000. Pour mesurer l'influence de cette incertitude, on considère la configuration optimale trouvée par la stratégie d'évolution. On réalise un échantillonnage de taille 100 de la charge et on calcule pour chaque charge l'effort mécanique interne. On constate sur la figure 11 que pour un nombre important d'échantillons la valeur de 400 est dépassée.

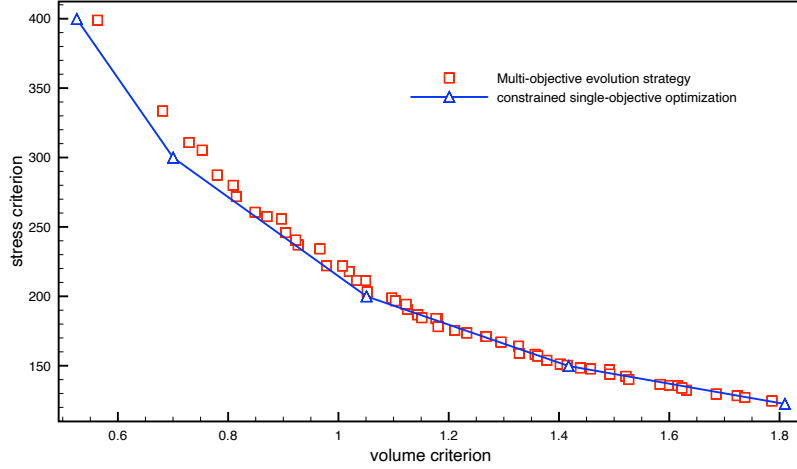


Figure 10: Front de Pareto par variation de la contrainte et une approche évolutionnaire.

Pour y remédier, on adopte une stratégie d'optimisation du pire cas. On ré-utilise alors la stratégie d'évolution, mais avec une contrainte correspondant au pire cas trouvé sur un échantillonnage de taille 100 de la charge. Ainsi, dès qu'une charge dans l'échantillon conduit à un dépassement de la valeur 400 de l'effort interne, la pénalisation devient active. Comme on peut le constater sur la figure 11, l'optimum est bien plus satisfaisant car pour la configuration optimale trouvée aucun cas de l'échantillon ne dépasse la valeur 400. Cependant, l'optimum correspond alors à un volume supérieur (0.67837 contre 0.55659 pour le problème sans incertitude).

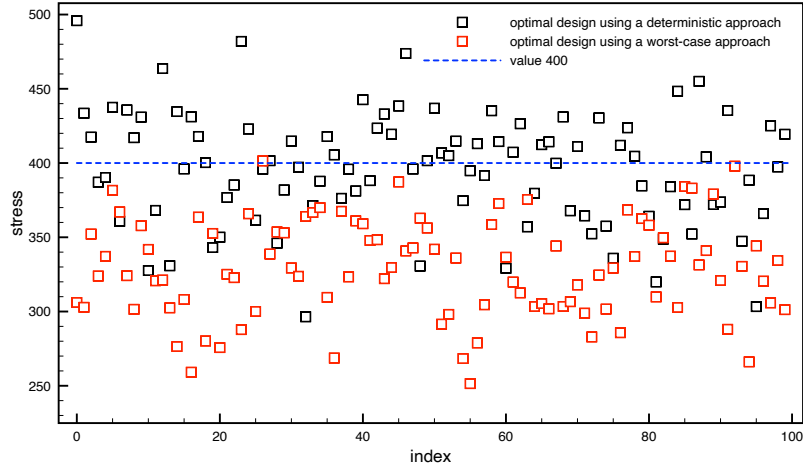


Figure 11: Echantillonnage de l'effort mécanique interne, pour l'optimum déterministe et un optimum basé sur le pire cas

References

- [1] E.H.L. Aarts and K. Korst. *Simulated annealing and Boltzmann machines*. John Wiley, 1989.
- [2] A. Conn, K. Scheinberg, and Ph.L. Toint. Recent progress in unconstrained nonlinear optimization without derivatives. *Mathematical Programming*, 79:397–414, 1997.
- [3] A. Conn, K. Scheinberg, and Ph.L. Toint. the convergence of derivative free methods for unconstrained optimization, 1997.
- [4] J.E. Dennis and V. Torczon. Direct search methods on parallel machines. *SIAM Journal of Optimization*, 1(4):448–474, 1991.
- [5] M. Dorigo and T. Stutzle. *Ant Colony Optimization*. MIT Press, 2004.
- [6] R. Fletcher. *Practical Methods of Optimization*. John Wiley & Sons, 1987.
- [7] P. E. Gill, W. Murray, and M. H. Wright. *Practical Optimization*. Academic Press, 1981.
- [8] D.E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison Wesley Company Inc., 1989.
- [9] N. Hansen, S.D. Muller, and P. Koumoutsakos. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (cma-es). *Evolutionary Computation*, 11(1):1–18, 2003.
- [10] J. Holland. *Adaptation in natural and artificial systems*. University of Michigan Press, 1975.
- [11] A. Jameson, L. Martinelli, and N. A. Pierce. Optimum aerodynamic design using the Navier-Stokes equation. *Theoretical and Computational Fluid Dynamics*, 10:213–237, 1998.
- [12] D.R. Jones. A taxonomy of global optimization methods based on response surfaces. *Journal of Global Optimization*, 21:345–383, 2001.
- [13] J. Kennedy and R.C. Eberhart. Particle swarm optimization. In *1995 IEEE International Conference on neural networks, Perth, Australia*, 1995.
- [14] M. Marazzi and J. Nocedal. Wedge trust region methods for derivative free optimization. *Mathematical Programming*, 91(2):289–305, 2002.
- [15] Z. Michalewics. *Genetic algorithms + data structures = evolutionary programs*. AI series. Springer-Verlag, New York, 1992.

- [16] B. Mohammadi and O. Pironneau. Mesh adaptation and automatic differentiation in a cad-free framework for optimal shape. *International Journal for Numerical Methods in Engineering*, 30(2):127–136, May 1999.
- [17] J. A. Nelder and R. Mead. A simplex method for function minimization. *Computer Journal*, 7:308–313, 1965.
- [18] M.J.D. Powell. An efficient method for finding the minimum of a function of several variables without calculating derivatives. *The Computer Journal*, 7, 1964.
- [19] M.J.D. Powell. A direct search optimization method that models the objective and constraint functions by linear interpolation. in S. Gomez and J-P. Hennart, editors, *Advances in Optimization and Numerical Analysis*, pp 51–67, printed in the Netherlands, 1994.
- [20] M.J.D. Powell. A direct search optimization method that models the objective by quadratic interpolation. Presentation at the 5th Stockholm Optimization Days, 1994.
- [21] H. H. Rosenbrock. An automatic method for finding the greatest or least value of a function. *Computer Journal*, 3:175–184, 1960.
- [22] W. Spendley, G. R. Hext, and F. R. Himsworth. Sequential application of simplex designs in optimisation and evolutionary operation. *Technometrics*, 4:441–461, 1962.
- [23] N. Srinivas and K. Deb. Multiobjective optimization using non-dominated sorting in genetic algorithm. *Evolutionary Computing*, 3(2):221–248, 1995.
- [24] W. H. Swann. Report on the development of a new direct search method of optimization. Technical Report Research Note 64/3, ICI Central Instrument Laboratory, 1964.
- [25] V. Torczon. *Multi-Directional Search: A Direct Search Algorithm for Parallel Machines*. PhD thesis, Houston, TX, USA, 1989.
- [26] D. Wienfield. Function minimization by interpolation in a data table. *Journal of the Institute of Mathematic Applications*, 12:339–347, 1973.